图论

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 v

第一章 图论相关概念 1

1.1 函数图的入环链与到达查询 . . . 1

1.2 杭州 2023 H：随机事件顺序与函数图 . . . 2

第二章 树上问题 3

2.1 树的直径 . . . 3

2.1.1 边权修改动态树直径 . . . 3

2.1.2 树上点集直径摘要 . . . 8

2.1.3 非负权树的点集直径摘要 . . . 9

2.2 最近公共祖先 . . . 10

2.2.1 两条树路径的交集 . . . 10

2.2.2 倍增 LCA、祖先与最大边权 . . . 11

2.2.3 树路径交：投影与单点交 . . . 14

2.3 树链剖分 . . . 14

2.3.1 长链剖分与常数时间 K 级祖先 . . . 14

2.3.2 重链剖分与 LCA . . . 19

2.4 树上启发式合并 . . . 24

2.4.1 树上启发式合并：子树众数颜色和 . . . 24

2.5 虚树 . . . 26

2.5.1 虚树：关键点压缩与路径信息 . . . 26

2.6 树分治 . . . 29

2.6.1 点分治：树上距离点对计数 . . . 29

2.6.2 点分治、启发式合并与虚树用法 . . . 32

2.7 动态树分治 . . . 32

2.7.1 动态点分树：点权赋值与距离球求和 . . . 32

2.7.2 动态点分树：有符号边权活动点直径与端点 . . . 37

2.8 树哈希 . . . 42

2.8.1 确定性有根与无根树同构 . . . 42

第三章 最短路问题 45

3.1 最短路 . . . 45

3.1.1 非负权最短路 . . . 45

3.1.2 Bellman–Ford：负环影响与路径方案 . . . 46

3.1.3 SPFA：队列松弛、负环判定与有限路径 . . . 48

3.1.4 Floyd 全源最短路与原边方案 . . . 50

3.1.5 杭州 2023 G：开放时间的单位边最短路 . . . 52

3.2 k 短路 . . . 53

3.2.1 非负权严格次短行走 . . . 53

3.2.2 正权前 K 短行走 . . . 55

第四章 生成树问题 **57**
4.1 最小生成树 . . . 57
4.1.1 Kruskal 重构森林与阈值连通块 . . . 57
4.1.2 曼哈顿最小生成树 . . . 60
4.1.3 严格与非严格次小生成树、换边方案 . . . 63
4.1.4 Kruskal 最小生成树与生成森林 . . . 67
4.1.5 稠密图 Prim 最小生成森林 . . . 69
4.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 71
4.1.7 严格次小生成树与换边 . . . 71
4.2 最小树形图 . . . 71
4.2.1 有向最小树形图 . . . 71

第五章 斯坦纳树 **75**
5.1 最小斯坦纳树与原边方案 . . . 75

第六章 连通性相关 **81**
6.1 强连通分量 . . . 81
6.1.1 函数图分解、跳转与到达 . . . 81
6.1.2 Tarjan 强连通分量与缩点 . . . 84
6.1.3 SCC 模板题与速度榜口径 . . . 86
6.2 双连通分量 . . . 87
6.2.1 点双与边双：共享 lowlink 核心 . . . 87
6.2.2 边双缩点森林（保留桥编号） . . . 90
6.3 圆方树 . . . 91
6.3.1 静态仙人掌森林最短路 . . . 91
6.3.2 由点双构造圆方森林 . . . 96

第七章 环计数问题 **97**
7.1 简单无向图四元环计数 . . . 97
7.2 无向简单图三元环枚举 . . . 98

第八章 最小环 **100**
8.1 有向图最小平均权环：线性空间 Karp . . . 100
8.2 非负权无向图最小环与原边方案 . . . 102

第九章 2-SAT **107**
9.1 2-SAT 与任意可行方案 . . . 107
9.2 2-SAT 编号方向与双连通定义 . . . 108

第十章 欧拉图 **109**
10.1 有向图字典序最小欧拉路 . . . 109
10.2 无向图欧拉路与边方向 . . . 112
10.3 欧拉回路与指定端点定向 . . . 115
10.4 任意起终点的混合欧拉路径 . . . 117

第十一章 网络流 **118**
11.1 最大流 . . . 118
11.1.1 HLPP 最高标号预流推进与流方案 . . . 118
11.1.2 最大流与方案 . . . 123
11.1.3 排列约束与最大流单位边分类 . . . 127

11.2 最小割 . . . 128
11.2.1 最大权闭合子图与选点方案 . . . 128
11.2.2 正容量最小割的可行边与必选边 . . . 129
11.2.3 Gomory–Hu 最小割树 . . . 130
11.2.4 闭合集、树形图与全局割的完整用法 . . . 131
11.3 费用流 . . . 132
11.3.1 势能最小费用最大流 . . . 132
11.3.2 允许负环的最小费用最大流 . . . 136
11.3.3 SPFA 费用流与残量方案 . . . 138
11.4 上下界网络流 . . . 139
11.4.1 上下界可行循环流 . . . 139
11.4.2 有源汇上下界最值流与边流量 . . . 141
11.5 Stoer–Wagner 算法 . . . 142
11.5.1 无向全局最小割 . . . 142

第十二章 图的匹配 145
12.1 二分图最大匹配 . . . 145
12.1.1 正则二分图完美匹配 . . . 145
12.1.2 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 148
12.1.3 二分图最大匹配的可行边与必选边 . . . 152
12.1.4 二分图最大独立集方案 . . . 153
12.2 二分图最大权匹配 . . . 153
12.2.1 矩形二分图最小权匹配 . . . 153
12.3 一般图最大匹配 . . . 155
12.3.1 一般图最大基数匹配：带花树 . . . 155
12.4 一般图最大权匹配 . . . 159
12.4.1 一般图最大权匹配：带权带花树 . . . 159

第十三章 Prüfer 序列 169
13.1 Prüfer 双射与有标号树计数 . . . 169
13.1.1 双射为何成立 . . . 169
13.1.2 Cayley、指定度数与叶子数 . . . 169
13.1.3 加权 Cayley 与连接已有连通块 . . . 170

第十四章 矩阵树定理 171
14.1 Tutte 多项式求值 . . . 171
14.2 任意正模数带权矩阵树 . . . 176
14.3 带权矩阵树定理与方向约定 . . . 178
14.4 任意正模数的矩阵树计数 . . . 179

第十五章 LGV 引理 180
15.1 LGV：DAG 路径权值行列式 . . . 180

第十六章 支配树 182
16.1 支配树：Lengauer–Tarjan . . . 182

第一部分 跨类依赖 **185**

附录 A 数论 **186**
A.1 模算术简介 . . . 186
A.1.1 静态模整数 . . . 186
A.2 卢卡斯定理 . . . 189
A.2.1 Lucas：小素数模的大下标组合数 . . . 189

附录 B 组合数学 **191**
B.1 排列组合 . . . 191
B.1.1 阶乘组合数 . . . 191

附录 C 线性代数 **192**
C.1 行列式 . . . 192
C.1.1 任意正模数行列式：欧几里得消元 . . . 192

附录 D 并查集 **194**
D.1 并查集 . . . 194
D.1.1 并查集 . . . 194

附录 E 树状数组 **196**
E.1 树状数组第 k 小与组合依赖 . . . 196

附录 F 线段树 **198**
F.1 线段树基础 . . . 198
F.1.1 通用线段树与左右边界搜索 . . . 198

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　图论相关概念

1.1　函数图的入环链与到达查询

代码见第 81 页。接口 0-based：题目 1-based 时构造减 1，只有跳转返回的顶点加 1；最少步数的 0 与不可达的 -1 原样输出。

每个弱连通分量是一个环加若干指向环的树。入度剥离后剩下环点；反向处理剥离序列即可得到环编号、入环点和入环距离。跳转 k 步：k 小于入环距离时倍增，否则先到入环点，余步对环长取模；k 可达无符号 64 位上界，倍增表 $O(\log n)$ 层。u 到 v 最少步数：环编号不同则不可达；v 在树上时要求 $\mathrm{depth}(u) \geq \mathrm{depth}(v)$ 且跳深度差恰到 v；v 在环上时为 u 的入环步数加环上顺向距离。不可达返回 -1，自身为 0。position 只对环点有意义，树点为 -1。

1.2 杭州 2023 H：随机事件顺序与函数图

QOJ8237。事件 u 的目标 $v = b_u$：$a_u < a_v$ 必成功，$a_u \geq a_v + w_v$ 必失败，否则 u 成功当且仅当 v 更早执行且成功。确定事件的后继改为自身。到达确定失败终点或纯依赖环（含自环）则概率为零；到达确定成功终点、入环距离 d 时概率为 $1/(d+1)!$，期望 $a_u + w_u/(d+1)!$，模 10^9+7。依赖不可当作独立的 $1/2$。$\sum n \leq 500000$，阶乘表跨组只扩不缩；用通用函数图时 $O(n \log n)$ 时间与空间。

第二章　树上问题

2.1　树的直径

相关应用：固定拓扑下修改非负边权，递归欧拉序与区间加维护动态直径。

2.1.1　边权修改动态树直径

DynamicDiameter(n)，n≥1，点号 1..n。add(u,v,w) 返回从 0 开始的原边编号；输入须恰为一棵非负权无向树。build(root=1) 预处理；update(id,w) 设置原边权值；diameter() 返回最长路径长度，单点树为 0。仅返回长度，不支持 link/cut 和负权。可换根重建、复制；所有内部成员仅供只读。全部边权之和始终 ≤LLONG_MAX/4，保证中间值安全。

递归 DFS 记录进入顶点及每次孩子返回后的根距离，得到 2n-1 项欧拉序。修改父边等价于对子树首次至最后出现的闭区间加 Δ。维护 max(d[i]-2d[j]+d[k])，i≤j≤k；非负权下区间最小根距离等于 LCA 根距离，故最大值即直径。线段树五项 high、low、left、right、best 分别表示 d、-2d、前两项、后两项、完整三项最大值。统一加 Δ 后依次加 Δ、-2Δ、-Δ、-Δ、0。建树 O(n)，修改 O(log n)，查询 O(1)，空间 O(n)。DFS 保持递归，深链须给足原生栈。LOJ3163 已满分通过，排名未核验。

字段、加边与五项合并：

```
struct DynamicDiameter
{
    using ll = long long;
    struct Node
    {
        ll high, low, left, right, best, lazy;
    };
    struct Edge
    {
        int u, v;
        ll w;
    };
    int n, size = 0;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;
    vector<int> first, last, child;
    vector<Node> t;

    explicit DynamicDiameter(int n) : n(n), g(n + 1), first(n + 1), last(n + 1)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0 && w <= LLONG_MAX / 4 - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        size = 0;
        return id;
    }
    static Node merge(const Node &a, const Node &b)
    {
        Node c;
        c.high = max(a.high, b.high);
        c.low = max(a.low, b.low);
        c.left = max({a.left, b.left, a.high + b.low});
        c.right = max({a.right, b.right, a.low + b.high});
        c.best = max({a.best, b.best, a.left + b.high, a.high + b.right});
        c.lazy = 0;
        return c;
    }
```

递归欧拉遍历与建树（接上页结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n && (int)edges.size() == n - 1);
        vector<ll> d;
        child.resize(edges.size());
        auto dfs = [&](auto &&self, int u, int parent, ll depth) -> void
        {
            first[u] = d.size();
            d.push_back(depth);
            for (auto [v, id] : g[u])
            {
                if (v == parent) continue;
                child[id] = v;
                self(self, v, u, depth + edges[id].w);
                d.push_back(depth);
            }
            last[u] = d.size() - 1;
        };
        dfs(dfs, root, 0, 0);
        size = d.size();
        t.resize(4 * size);
        auto init = [&](auto &&self, int x, int l, int r) -> void
        {
            if (l == r)
            {
                t[x] = {d[l], -2 * d[l], -d[l], -d[l], 0, 0};
                return;
            }
            int mid = (l + r) / 2;
            self(self, x * 2, l, mid);
            self(self, x * 2 + 1, mid + 1, r);
            t[x] = merge(t[x * 2], t[x * 2 + 1]);
        };
        init(init, 1, 0, size - 1);
    }
```

区间加与原边修改（接上页结构体）：

```
    void apply(int x, ll delta)
    {
        t[x].high += delta;
        t[x].low -= 2 * delta;
        t[x].left -= delta;
        t[x].right -= delta;
        t[x].lazy += delta;
    }
    void change(int x, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr)
        {
            apply(x, delta);
            return;
        }
        apply(x * 2, t[x].lazy);
        apply(x * 2 + 1, t[x].lazy);
        t[x].lazy = 0;
        int mid = (l + r) / 2;
        if (ql <= mid) change(x * 2, l, mid, ql, qr, delta);
        if (qr > mid) change(x * 2 + 1, mid + 1, r, ql, qr, delta);
        t[x] = merge(t[x * 2], t[x * 2 + 1]);
    }
    void update(int id, ll w)
    {
        assert(size && 0 <= id && id < (int)edges.size());
        assert(w >= 0 && w <= LLONG_MAX / 4 - (total - edges[id].w));
        ll delta = w - edges[id].w;
        total += delta;
        edges[id].w = w;
        int u = child[id];
        change(1, 0, size - 1, first[u], last[u], delta);
    }
    ll diameter() const
    {
        assert(size);
        return t[1].best;
    }
};
```

模板题使用：CEOI2019 Dynamic Diameter（LOJ3163 待复核提交）

题意：每次修改一条边后输出直径；用上次答案解码边号和新权，边编号 0-based。

所需模板：DynamicDiameter（第 3 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/3163

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w;
    cin >> n >> q >> w;
    DynamicDiameter tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        tree.add(u, v, c);
    }
    tree.build();
    long long last = 0;
    while (q--)
    {
        long long d, e;
        cin >> d >> e;
        int id = (d + last) % (n - 1);
        tree.update(id, (e + last) % w);
        last = tree.diameter();
        cout << last << '\n';
    }
}
```

2.1.2 树上点集直径摘要

默认空点集，a=b=-1、length=0。insert(v,distance) 插入顶点，重复插入合法；merge(other,distance) 合并两个点集摘要，允许自合并；farthest(v,distance) 返回 optional<pair<int,long long»（最远顶点、距离），空集返回 nullopt，距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离，点号为非负整数；所有距离 ≤LLONG_MAX。每次插入或最远查询最多两次距离调用，合并最多四次，摘要只占 O(1) 空间。保存 a、b 为点集中某对直径端点，不保留全部点；字段只读，不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合，重置用默认构造。

```
struct TreeDiameter
{
    using ll = long long;
    int a = -1, b = -1;
    ll length = 0;

    // distance is the metric of one fixed tree with nonnegative edge weights.
    template <class Distance> void insert(int v, const Distance &distance)
    {
        assert(v >= 0);
        if (a == -1)
        {
            a = v;
            b = v;
            return;
        }
        ll x = distance(a, v), y = distance(b, v);
        if (x >= y && x > length)
        {
            b = v;
            length = x;
        }
        else if (y > length)
        {
            a = v;
            length = y;
        }
    }
```

合并与最远点查询（接上页同一结构体）：

```
    template <class Distance> void merge(TreeDiameter other, const Distance &distance)
    {
        if (other.a == -1) return;
        insert(other.a, distance);
        insert(other.b, distance);
    }

    template <class Distance>
    optional<pair<int, ll>> farthest(int v, const Distance &distance) const
    {
        if (a == -1) return nullopt;
        ll x = distance(a, v), y = distance(b, v);
        return x >= y ? make_pair(a, x) : make_pair(b, y);
    }
};
```

2.1.3 非负权树的点集直径摘要

代码见第 8 页。非空点集只需保存一对直径端点 a, b，任意点 x 到点集的最远距离为 $\max(d(x,a), d(x,b))$。要求边权非负（可为零），不能推广到一般图或负权树。插入新点只比较到两个旧端点的距离；合并点集只插入另一摘要的两个端点。并列直径任选，端点不固定。带权距离可用 $(D_u - D_z) + (D_v - D_z)$（$z$ 为 LCA）；所有路径权和须在 long long 内。

离线添叶仍是固定树度量。 CF379F：先建出最终树再预处理距离，按出现顺序向摘要插点（初始四点也插入）。$q \le 500000$ 时 $4 + 2q$ 个点，HLD $O(q)$ 空间、$O(q \log q)$ 时间。

QOJ8235（点权互异）：令 g 为全树点权的 mex，按点权 $0, \ldots, g-1$ 建前缀摘要；某前缀全在以 x 为中心、半径 k 的范围内当且仅当其最远距离 $\le k$，二分最长前缀；$g = 0$ 时为零。边带权，不能用边数距离。

2.2 最近公共祖先

2.2.1 两条树路径的交集

path_intersection(a,b,c,d,depth,lca) 返回路径 a-b 与 c-d 的公共路径，字段 u、v 是交集端点，vertices 是公共顶点数；空交返回 {0,0,0}，单点交的 u=v。点号从 1 开始，depth 数组与 lca 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 vector 深度表，回调接收两个点并返回 LCA。将 c、d 分别投影到 a-b 得 p、q；投影是 lca(a,b)、lca(a,x)、lca(b,x) 中最深者。若 p≠q，c-d 必经过 p-q；若 p=q，检验 p 是否在 c-d 上即可区分单点交与空交。不依赖顶点编号大小。常数次 LCA 调用、O(1) 额外空间；使用 EulerLCA 时为 O(1)，HLD/倍增时为 O(log n)。公共边数为 max(0,vertices-1)。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
struct PathIntersection
{
    int u, v, vertices;
};

// Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
template <class Depth, class Lca>
PathIntersection
path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
{
    int z = lca(a, b);
    auto project = [&](int x)
    {
        int p = z;
        for (int v : {lca(a, x), lca(b, x)})
            if (depth[v] > depth[p]) p = v;
        return p;
    };
    auto distance = [&](int u, int v)
    {
        return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
    };
    int p = project(c), q = project(d);
    if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
    return {p, q, int(distance(p, q) + 1)};
}
```

2.2.2 倍增 LCA、祖先与最大边权

构造 LiftingLCA(n)，add(u,v,w=1) 加边，build(root=1) 预处理非空无向树，点号 1..n。jump(u,k) 返回 k 级祖先，k 非负，超过根返回 0；lca(u,v) 求最近公共祖先；distance 返回路径边数，不是权值和；max_edge 返回路径最大边权，同点返回 nullopt，其他情况返回有值的 optional<long long>。支持负权和 LLONG_MIN，LOW 只是最大值合并的单位元，不表示空路径。DFS 递归，根深度为 0；up[k][u] 与 mx[k][u] 是 2 的 k 次方长度向上段的祖先与最大边权，0 号节点为哨兵。可重复 build 换根；修改图后必须重建。预处理 O(n log n) 时间与空间，各查询 O(log n)。不需要边权最值时，可删去 mx 和 max_edge 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证；P3379、POJ1330 驱动只验证 LCA，最大边权仍是本地证据。

```
struct LiftingLCA
{
    using ll = long long;
    static constexpr ll LOW = numeric_limits<ll>::lowest();
    int n, levels;
    bool ready = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mx;
    vector<int> depth;

    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
    {
        assert(n > 0);
        levels = 1;
        while ((1LL << levels) <= n) levels++;
        up.assign(levels, vector<int>(n + 1));
        mx.assign(levels, vector<ll>(n + 1, LOW));
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        ready = false;
    }
```

```
    void dfs(int u, int parent, ll weight)
    {
        up[0][u] = parent;
        mx[0][u] = weight;
        for (int k = 1; k < levels; k++)
        {
            int p = up[k - 1][u];
            up[k][u] = up[k - 1][p];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
        }
        for (auto [v, w] : g[u])
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                dfs(v, u, w);
            }
    }

    // Nonempty tree only. Node 0 means no ancestor.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        depth[root] = 0;
        dfs(root, 0, LOW);
        ready = true;
    }

    int jump(int u, long long steps) const
    {
        assert(ready && 1 <= u && u <= n && steps >= 0);
        if (steps > depth[u]) return 0;
        for (int k = 0; k < levels; k++)
            if (steps >> k & 1) u = up[k][u];
        return u;
    }
```

```
    int lca(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (depth[u] < depth[v]) swap(u, v);
        u = jump(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        return up[0][u];
    }

    int distance(int u, int v) const
    {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }

    optional<ll> max_edge(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return nullopt;
        if (depth[u] < depth[v]) swap(u, v);
        ll answer = LOW;
        int delta = depth[u] - depth[v];
        for (int k = 0; k < levels; k++)
            if (delta >> k & 1)
            {
                answer = max(answer, mx[k][u]);
                u = up[k][u];
            }
        if (u == v) return answer;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                answer = max(answer, max(mx[k][u], mx[k][v]));
                u = up[k][u];
                v = up[k][v];
            }
        return max(answer, max(mx[0][u], mx[0][v]));
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 11 页，2.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    LiftingLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

2.2.3 树路径交：投影与单点交

接口见第 10 页。x 在路径 a–b 上的投影是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者（同一根）。设 c,d 的投影为 p,q：$p\neq q$ 时交集为 p–q；$p=q$ 时若 $d(c,p)+d(p,d)=d(c,d)$ 则为单点交，否则为空。距离与 depth 按边数，不能换成带权根距离。返回交集端点和顶点数，空交为 $\{0,0,0\}$，公共边数 $\max(0,\text{vertices}-1)$。每次调用常数次 LCA；用 HLD 回调为 $O(\log n)$。

2.3 树链剖分

2.3.1 长链剖分与常数时间 K 级祖先

LongChain(n)，1≤n<INT_MAX，点号 1..n；add(u,v) 加入无向树边，调用方保证全部边恰好构成一棵树，不支持环、重边、自环或森林。build(root=1) 以指定根重建，dep[root]=1；kth(u,k) 返回 k 级祖先，k≥0，k=0 返回自身，越过根返回 0。查询前必须 build，加边后必须重建；支持重复 build、换根与值复制。son 按子树最大高度选长儿子，不能换成子树大小；len 含自身。链顶 t 的 down[t] 从 t 向下，up[t][0]=t 且向上越根补 0；成员只读，dfs1/dfs2 仅内部使用。预处理 O(n log(n+1))，空间 O(n log(n+1))，两次 DFS 均递归、栈深 O(n)。查询先跳最高 2 的幂，再在链顶上下表中查一次，32 位字长模型下 O(1)；k=0 必须先处理，不能求 0 的最高位。

```
struct LongChain
{
    int n;
    vector<int> dep, len, son, top;
    vector<vector<int>> g, fa, up, down;
```

```
6
7     LongChain(int n) : n(n), g(n + 1)
8     {
9         assert(1 <= n && n < INT_MAX);
10    }
11
12    void add(int u, int v)
13    {
14        g[u].push_back(v);
15        g[v].push_back(u);
16    }
17
18    void dfs1(int u, int p)
19    {
20        fa[0][u] = p;
21        dep[u] = dep[p] + 1;
22        for (int j = 1; j < (int)fa.size(); j++)
23            fa[j][u] = fa[j - 1][fa[j - 1][u]];
24        for (int v : g[u])
25        {
26            if (v == p) continue;
27            dfs1(v, u);
28            if (len[v] + 1 > len[u])
29            {
30                len[u] = len[v] + 1;
31                son[u] = v;
32            }
33        }
34    }
```

递归划分长链并填表（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        if (u == t)
        {
            down[t].reserve(len[u]);
            up[t].resize(len[u]);
            int x = u;
            for (int &v : up[t])
            {
                v = x;
                x = fa[0][x];
            }
        }
        down[t].push_back(u);
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[0][u] && v != son[u])
                dfs2(v, v);
        }
    }
```

重建与常数时间查询（接上页同一结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        dep.assign(n + 1, 0);
        len.assign(n + 1, 1);
        son.assign(n + 1, 0);
        top.assign(n + 1, 0);
        fa.assign(bit_width((unsigned)n), vector<int>(n + 1));
        up.assign(n + 1, {});
        down.assign(n + 1, {});
        dfs1(root, 0);
        dfs2(root, root);
    }

    int kth(int u, int k) const
    {
        assert(1 <= u && u <= n && k >= 0 && !fa.empty());
        if (k >= dep[u]) return 0;
        if (k == 0) return u;
        int j = bit_width((unsigned)k) - 1;
        u = fa[j][u];
        k -= 1 << j;
        int t = top[u];
        int d = dep[u] - dep[t];
        if (k <= d) return down[t][d - k];
        return up[t][k - d];
    }
};
```

为什么一次跳跃就够 先向上跳 p（不超过 k 的最大 2 的幂），到达 u，剩余 r 小于 p。原查询点是 u 下方距离 p 的后代，所以 u 的子树高度至少 p+1。u 所在长链顶 t 的链长至少 dep[u]-dep[t]+p+1，足以覆盖 r；若 r 不超过 u 到 t 的距离就查 down，否则查 up。选择长儿子必须按高度，不能按子树大小。

边界与空间 up[t][0] 和 down[t][0] 都等于 t；up 越过根填 0。先处理 k=0 和越根，避免对 0 求最高位。各长链长度之和为 n，上下表各存 n 个 int；倍增表为 O(n log n)。预处理两次 DFS 都保持递归，栈深可达 n。

模板题使用：P5903 【模板】树上 K 级祖先

题意：按题意用 32 位无符号 xorshift 顺序生成询问；根深度为 1，先生成 x 再生成 k，按 64 位累计 i* 答案的异或；n≤500000，q≤5000000。

所需模板：LongChain（第 14 页，2.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5903

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    uint32_t s;
    cin >> n >> q >> s;
    LongChain tree(n);
    int root = 0;
    for (int u = 1; u <= n; u++)
    {
        int p;
        cin >> p;
        if (p) tree.add(u, p);
        else root = u;
    }
    tree.build(root);
    auto get = [&]()
    {
        s ^= s << 13;
        s ^= s >> 17;
        s ^= s << 5;
        return s;
    };
    int last = 0;
    unsigned long long ans = 0;
    for (int i = 1; i <= q; i++)
    {
        int x = (get() ^ last) % n + 1;
        int k = (get() ^ last) % tree.dep[x];
        last = tree.kth(x, k);
        ans ^= 1ULL * i * last;
    }
    cout << ans << '\n';
}
```

2.3.2 重链剖分与 LCA

add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 19 页，2.3.2 节）、segtree（第 198 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

2.4 树上启发式合并

2.4.1 树上启发式合并：子树众数颜色和

SubtreeColors(n)，点号 1..n，add(u,v) 建无向树；要求图连通且无环，n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组，colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和，max_freq[u] 为该频次；支持负颜色与稀疏编号，内部离散化，和用 long long。重复 run 可更换颜色或根。重儿子信息保留，轻儿子通过 DFS 序区间加入；清空整棵临时子树后再重置最大值，不尝试逐次删除时维护众数。O(n log n) 时间、O(n) 空间，全部 DFS 递归。insert 是内部增量统计，可按题目修改统计量，但相应清空规则也必须调整。

```
struct SubtreeColors
{
    int n, best = 0;
    long long sum = 0;
    vector<vector<int>> g;
    vector<int> siz, heavy, tin, tout, order, color, value, freq;
    vector<int> max_freq;
    vector<long long> answer;

    SubtreeColors(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void prepare(int u, int p)
    {
        siz[u] = 1;
        tin[u] = order.size();
        order.push_back(u);
        for (int v : g[u])
        {
            if (v == p) continue;
            prepare(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
        }
        tout[u] = (int)order.size() - 1;
    }

    void insert(int u)
    {
        int c = color[u];
        freq[c]++;
        if (freq[c] > best)
        {
            best = freq[c];
            sum = value[c];
        }
        else if (freq[c] == best)
            sum += value[c];
    }
```

启发式合并与运行入口（接上页同一结构体）：

```
    void solve(int u, int p, bool keep)
    {
        for (int v : g[u])
        {
            if (v != p && v != heavy[u]) solve(v, u, false);
        }
        if (heavy[u]) solve(heavy[u], u, true);
        for (int v : g[u])
        {
            if (v == p || v == heavy[u]) continue;
            for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
        }
        insert(u);
        max_freq[u] = best;
        answer[u] = sum;
        if (!keep)
        {
            for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
            best = 0;
            sum = 0;
        }
    }

    // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
    void run(const vector<int> &colors, int root = 1)
    {
        assert((int)colors.size() == n + 1);
        siz.assign(n + 1, 0);
        heavy = siz;
        tin = siz;
        tout = siz;
        color = siz;
        max_freq = siz;
        answer.assign(n + 1, 0);
        order.clear();
        value.assign(colors.begin() + 1, colors.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        freq.assign(value.size(), 0);
        best = 0;
        sum = 0;
        if (!n) return;
        assert(1 <= root && root <= n);
        for (int u = 1; u <= n; u++)
            color[u] =
                lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
        prepare(root, 0);
        solve(root, 0, false);
    }
};
```

2.5 虚树

2.5.1 虚树：关键点压缩与路径信息

VirtualTree(n)，n≥1，点号 1..n；add(u,v,w=1) 建非负权连通树，无环和重边。prepare(root=1) 递归预处理，可换根重建；根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA，返回 vertices 和 edges；全部编号沿用原树，vertices 按 DFS 序排列，非空时首项即虚树根。edges 为 (from,to,distance,minimum)，分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果，单点返回单节点；原树根不会强制加入，需要时由调用者加入 keys。边按后代 DFS 序排列，可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表；每次仅返回 O(k) 结果。预处理 O(n log n) 时间空间，单次 O(k log k+k log n)。lca、ancestor 也可直接使用，要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法，原树 DFS 保持递归。

```
struct VirtualTree
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll distance, minimum;
    };

    struct Result
    {
        vector<int> vertices;
        vector<Edge> edges;
    };

    int n, log = 1, timer = 0;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    vector<int> depth, tin, tout;
    vector<ll> dist;

    VirtualTree(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

预处理与最近公共祖先（接上页同一结构体）：

```
    void dfs(int u, int p)
    {
        tin[u] = ++timer;
        up[0][u] = p;
        for (int j = 1; j < log; j++)
        {
            up[j][u] = up[j - 1][up[j - 1][u]];
            mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
        }
        for (auto [v, w] : g[u])
        {
            if (v == p) continue;
            assert(dist[u] <= numeric_limits<ll>::max() - w);
            depth[v] = depth[u] + 1;
            dist[v] = dist[u] + w;
            mn[0][v] = w;
            dfs(v, u);
        }
        tout[u] = timer;
    }

    void prepare(int root = 1)
    {
        assert(1 <= root && root <= n);
        log = 1;
        while ((1LL << log) <= n) log++;
        up.assign(log, vector<int>(n + 1));
        mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
        depth.assign(n + 1, 0);
        tin = depth;
        tout = depth;
        dist.assign(n + 1, 0);
        timer = 0;
        dfs(root, root);
    }

    bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const
    {
        if (ancestor(u, v)) return u;
        if (ancestor(v, u)) return v;
        for (int j = log - 1; j >= 0; j--)
        {
            if (!ancestor(up[j][u], v)) u = up[j][u];
        }
        return up[0][u];
    }
```

压缩路径与虚树构造（接上页同一结构体）：

```
    Edge path(int u, int v) const
    {
        assert(u != v && ancestor(u, v));
        ll minimum = numeric_limits<ll>::max();
        int x = v, steps = depth[v] - depth[u];
        for (int j = 0; j < log; j++)
        {
            if (steps >> j & 1)
            {
                minimum = min(minimum, mn[j][x]);
                x = up[j][x];
            }
        }
        return {u, v, dist[v] - dist[u], minimum};
    }

    // Uses original vertex IDs. Empty input gives an empty result.
    Result build(vector<int> keys) const
    {
        if (keys.empty()) return {};
        for (int u : keys) assert(1 <= u && u <= n);
        auto cmp = [&](int u, int v)
        {
            return tin[u] < tin[v];
        };
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        int k = keys.size();
        for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        Result result{keys, {}};
        vector<int> st;
        for (int u : keys)
        {
            while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
            if (!st.empty()) result.edges.push_back(path(st.back(), u));
            st.push_back(u);
        }
        return result;
    }
};
```

2.6 树分治

2.6.1 点分治：树上距离点对计数

CentroidPairs(n)，点号 1..n；add(u,v,w) 建树，允许森林但禁止环、重边、自环，边权非负；所有简单路径长度须能用 long long 表示。build() 预处理，可重复调用；改变边后须重建。count_leq(k) 统计距离 ≤k 的无序不同顶点对，count_exact(k) 统计距离恰为 k 的点对；负 k 返回 0，零距离只统计不同顶点，跨连通块不配对。parent[c] 是点分树父节点，根为 0。all[c] 为该层分治块到重心的距离，含重心距离 0；part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$，空间 $O(n \log n)$，每次计数 $O(n \log n)$；固定树上多次查询可以复用排序结果。DFS 递归，距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
struct CentroidPairs
{
    using ll = long long;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> siz, removed, parent;
    vector<vector<ll>> all;
    vector<vector<vector<ll>>> part;

    CentroidPairs(int n)
        : n(n),
          g(n + 1),
          siz(n + 1),
          removed(n + 1),
          parent(n + 1),
          all(n + 1),
          part(n + 1)
    {
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }
```

重心与分解（接上页同一结构体）：

```
    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
        {
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        }
        return u;
    }

    void collect(int u, int p, ll d, vector<ll> &out)
    {
        out.push_back(d);
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= numeric_limits<ll>::max() - w);
            collect(v, u, d + w, out);
        }
    }

    void decompose(int entry, int p)
    {
        size_dfs(entry, 0);
        int c = centroid(entry, 0, siz[entry]);
        removed[c] = 1;
        parent[c] = p;
        all[c] = {0};
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            part[c].push_back({});
            auto &d = part[c].back();
            collect(v, c, w, d);
            sort(d.begin(), d.end());
            all[c].insert(all[c].end(), d.begin(), d.end());
        }
        sort(all[c].begin(), all[c].end());
        for (auto [v, w] : g[c])
        {
            if (!removed[v]) decompose(v, c);
        }
    }
```

重建与距离计数（接上页同一结构体）：

```
    // Input must be a forest. Rebuild after changing edges.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            all[u].clear();
            part[u].clear();
        }
        for (int u = 1; u <= n; u++)
        {
            if (!removed[u]) decompose(u, 0);
        }
    }

    ll pairs(const vector<ll> &d, ll k) const
    {
        int l = 0, r = (int)d.size() - 1;
        ll answer = 0;
        while (l < r)
        {
            if (d[l] <= k - d[r])
            {
                answer += r - l;
                l++;
            }
            else
                r--;
        }
        return answer;
    }

    // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
    ll count_leq(ll k) const
    {
        if (k < 0) return 0;
        ll answer = 0;
        for (int c = 1; c <= n; c++)
        {
            answer += pairs(all[c], k);
            for (const auto &d : part[c]) answer -= pairs(d, k);
        }
        return answer;
    }

    ll count_exact(ll k) const
    {
        if (k < 0) return 0;
        return count_leq(k) - (k ? count_leq(k - 1) : 0);
    }
};
```

模板题使用：Luogu P3806

题意：正权树上查询距离为 k 的不同顶点对是否存在。先 build 一次，再用 count_exact 的结果是否非零输出 AYE/NAY。n 至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 29 页，2.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3806

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    CentroidPairs tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.build();
    while (m--)
    {
        long long k;
        std::cin >> k;
        std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
    }
}
```

2.6.2 点分治、启发式合并与虚树用法

点分治 count_exact 计无序不同顶点对，预处理 $O(n \log^2 n)$，每次查询 $O(n \log n)$；边修改后须重建。

启发式合并：轻子树先算后清空，重子树保留；并列最高频颜色编号之和须用 long long。

虚树：显式加入根后构造，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点取路径最小边权，否则取它与孩子 dp 的较小值（用 minimum，不用 distance）。

2.7 动态树分治

2.7.1 动态点分树：点权赋值与距离球求和

代码依赖：Fenwick（第 196 页，E.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

构造 CentroidSum(n)，T 默认为 long long；n≥1，输入是固定连通单位边树。点号 0..n-1；先 add(u,v)，再 build(values)。set(u,x) 是点权赋值；query(u,k) 返回到 u 距离 ≤k 的全部点权和，含 u 自身；负半径返回 0，半径用 long long。build 可在同一拓扑上重建并替换全部权值，建树后不再 add 边，不提供 link/cut。每点保存重心祖先、距离和分支，使用总距离桶减同支桶容斥；距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick，构造/空间 O(n log n)，修改/查询 O($\log^2$ n)。T 支持零值及加减，所有桶和、赋值差与容斥中间值均须可表示；可用负权，不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏 O(n)，不等于重心层数；最大链测试提供 256MiB 栈，仍需核对目标评测栈限制。

```
template <class T = long long> struct CentroidSum
{
```

```
    struct Record
    {
        int centroid, distance, branch;
    };
    int n;
    vector<vector<int>> g;
    vector<int> siz, removed;
    vector<T> value;
    vector<vector<Record>> path;
    vector<Fenwick<T>> all;
    vector<vector<Fenwick<T>>> part;

    CentroidSum(int n)
        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
    {
        assert(n > 0);
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

子树大小、重心与距离记录（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (int v : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (int v : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
    {
        path[u].push_back({c, d, b});
        if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
        bucket[d + 1] += value[u];
        for (int v : g[u])
            if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
    }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```
    // Raw distance d is stored at 1-based Fenwick index d+1.
    void make_bit(Fenwick<T> &bit, vector<T> bucket)
    {
        bit.n = (int)bucket.size() - 1;
        bit.a = move(bucket);
        for (int i = 1; i <= bit.n; i++)
            if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, 0, -1});
        vector<T> total{T{}, value[c]};
        for (int v : g[c])
        {
            if (removed[v]) continue;
            vector<T> bucket(1);
            collect(v, c, 1, c, (int)part[c].size(), bucket);
            if (total.size() < bucket.size()) total.resize(bucket.size());
            for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
            part[c].emplace_back();
            make_bit(part[c].back(), move(bucket));
        }
        make_bit(all[c], move(total));
        for (int v : g[c])
            if (!removed[v]) decompose(v);
    }
```

重建、点权赋值与距离球求和（接上页同一结构体）：

```
    // Rebuild resets all values and indices; do not change the tree after building.
    void build(const vector<T> &values)
    {
        assert((int)values.size() == n);
        value = values;
        fill(removed.begin(), removed.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            part[u].clear();
        }
        decompose(0);
    }

    void set(int u, T x)
    {
        assert(0 <= u && u < n);
        T delta = x - value[u];
        value[u] = x;
        for (auto [c, d, b] : path[u])
        {
            all[c].add(d + 1, delta);
            if (b != -1) part[c][b].add(d + 1, delta);
        }
    }

    T prefix(const Fenwick<T> &bit, long long radius) const
    {
        if (radius < 0) return T{};
        return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
    }

    T query(int u, long long radius) const
    {
        assert(0 <= u && u < n);
        T answer{};
        if (radius < 0) return answer;
        for (auto [c, d, b] : path[u])
        {
            answer += prefix(all[c], radius - d);
            if (b != -1) answer -= prefix(part[c][b], radius - d);
        }
        return answer;
    }
};
```

模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过 k 的权值和。操作 0 查询，操作 1 赋值；每次先将 x 及 y/k 与最近一次查询答案异或，再把 1-based 顶点 x 减一。更新不改变 last，操作类型不异或。n,m≤100000，点权/新值在 1..10000，解码后 k 在 0..n-1。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 196 页，E.1 节）、CentroidSum（第 32 页，2.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6329

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> value(n);
    for (auto &x : value) std::cin >> x;
    CentroidSum tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build(value);
    long long last = 0;
    while (m--)
    {
        int op;
        long long x, y;
        std::cin >> op >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 0)
        {
            last = tree.query((int)x - 1, y);
            std::cout << last << '\n';
        }
        else
            tree.set((int)x - 1, y);
    }
}
```

2.7.2 动态点分树：有符号边权活动点直径与端点

CentroidDiameter(n)，固定连通无向树、0-based 点号，n>0 且 2n 不溢出 int。add(u,v,w) 仅在首次 build 前；允许负边权，所有简单路径权值和必须能用 long long 表示。build() 递归分解并清空活动集，可反复重建；set(u,on) 幂等启停点。query() 返回 optional<tuple<long long,int,int»，内容为最大距离与任意一对达到最优值的活动端点，允许两端相同，所以非空集答案至少 0；空集 nullopt。各重心按不同分支取最大距离，只合并跨分支真实路径，不能套用非负权直径端点摘要；并列不承诺最小编号。空集预处理 O(n log n)、set $O(\log^2 n)$、query O(1)，空间 O(n log n)；逐点激活全树 $O(n \log^2 n)$。支持值拷贝；active 只读用于反色，其余字段与 sum/size_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归，树输入合法性与足够栈空间由调用方保证。

```cpp
struct CentroidDiameter
{
    using ll = long long;
    using Item = tuple<ll, int, int>;
    using Best = tuple<ll, int, int, int>;
    struct Entry
    {
        int c, b;
        ll d;
    };
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<Entry>> path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;
    vector<std::set<Item>> top;
    std::set<Best> best;
    std::set<int> live;

    CentroidDiameter(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
    {
        assert(0 < n && n <= INT_MAX / 2);
    }

    ll sum(ll a, ll b) const
    {
        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
        return a + b;
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

递归分解与不同分支的距离（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, ll d, int c, int b)
    {
        path[u].push_back({c, b, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            collect(v, u, sum(d, w), c, b);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, (int)bag.size(), 0});
        bag.emplace_back();
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            int b = bag.size();
            bag.emplace_back();
            collect(v, c, w, c, b);
        }
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建与重心候选（接上页同一结构体）：

```
    // Fixed tree, signed weights. Rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            top[u].clear();
        }
        bag.clear();
        best.clear();
        live.clear();
        decompose(0);
        built = true;
    }

    optional<Best> candidate(int c) const
    {
        if (top[c].size() < 2) return nullopt;
        auto it = top[c].rbegin();
        auto [x, u, a] = *it++;
        auto [y, v, b] = *it;
        if (u > v) swap(u, v);
        return Best{sum(x, y), u, v, c};
    }
```

活动点设置与端点查询（接上页同一结构体）：

```
    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        if (on) live.insert(u);
        else live.erase(u);
        for (auto [c, b, d] : path[u])
        {
            if (auto old = candidate(c)) best.erase(*old);
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].erase({x, v, b});
            }
            if (on) bag[b].insert({d, u});
            else bag[b].erase({d, u});
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].insert({x, v, b});
            }
            if (auto now = candidate(c)) best.insert(*now);
        }
    }

    // Maximum distance and any attaining pair; equal endpoints are allowed.
    optional<Item> query() const
    {
        assert(built);
        if (live.empty()) return nullopt;
        if (!best.empty())
        {
            auto [d, u, v, c] = *best.rbegin();
            if (d > 0) return Item{d, u, v};
        }
        int u = *live.begin();
        return Item{0, u, u};
    }
};
```

2.8 树哈希

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

2.8.1 确定性有根与无根树同构

TreeIsomorphism iso；rooted(g,root) 返回有根树编号，unrooted(g) 返回无根树 pair 编号。g 是 0-based 非空简单无向连通树，邻接表双向各存一次；root 在 0..n-1。递归 DFS，不验证非法图。所有待比较的树必须共用同一个 iso；编号依赖插入次序，不同对象或清空 codes 前后的编号不能直接比较。对子节点编号排序并用 map 精确驻留，无随机哈希碰撞。无根树取重心：单重心返回 (code,0)，双重心分别以两点为根编码并排序。支持单点；不考虑点/边标签，也不返回顶点映射。codes 只读，编号数须 <INT_MAX，n<=INT_MAX。S 为本次结束时驻留签名数，一次 n 点调用 O(n log(n+1)+n log(S+1))，临时 O(n)，跨调用累计存储至多 O(T)，T 为累计编码遍历的顶点数。

```
struct TreeIsomorphism
{
    map<vector<int>, int> codes;

    int rooted(const vector<vector<int>> &g, int root)
    {
        assert(0 <= root && root < int(g.size()));
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            vector<int> children;
            for (int v : g[u])
            {
                if (v == p) continue;
                children.push_back(self(self, v, u));
            }
            sort(children.begin(), children.end());
            int next = int(codes.size()) + 1;
            return codes.emplace(children, next).first->second;
        };
        return dfs(dfs, root, -1);
    }
```

继续求解（接上页同一结构体）：

```
    pair<int, int> unrooted(const vector<vector<int>> &g)
    {
        int n = g.size();
        assert(n > 0);
        vector<int> centers;
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            int size = 1, largest = 0;
            for (int v : g[u])
            {
                if (v == p) continue;
                int child = self(self, v, u);
                size += child;
                largest = max(largest, child);
            }
            largest = max(largest, n - size);
            if (largest <= n / 2) centers.push_back(u);
            return size;
        };
        dfs(dfs, 0, -1);
        int a = rooted(g, centers[0]);
        if (centers.size() == 1) return {a, 0};
        int b = rooted(g, centers[1]);
        if (a > b) swap(a, b);
        return {a, b};
    }
};
```

模板题使用：P5043 【模板】树同构 / [BJOI2015] 树的同构

题意：M 棵无根树，每棵 N 个点，1<=N,M<=50。输入 N 个父亲编号，0 表示根；转换无向树后忽略输入根。输出与本树同构的最早树编号；所有树共用同一驻留表。

所需模板：TreeIsomorphism（第 42 页，2.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    TreeIsomorphism iso;
    map<pair<int, int>, int> first;
    for (int id = 1; id <= m; id++)
    {
        int n;
        cin >> n;
        vector<vector<int>> g(n);
        for (int u = 0; u < n; u++)
        {
            int p;
            cin >> p;
            if (p == 0) continue;
            p--;
            g[u].push_back(p);
            g[p].push_back(u);
        }
        auto code = iso.unrooted(g);
        auto result = first.emplace(code, id);
        cout << result.first->second << '\n';
    }
}
```

第三章　最短路问题

3.1　最短路

3.1.1　非负权最短路

add(u,v,w)、run(s)，dis 为距离，pre 为前驱，path(t) 返回点序列。不可达用 LLONG_MAX；要求所有有限最短距离严格小于该哨兵，w≥0。懒删除堆在多重边图中按 O(n+m log(m+1)) 计，空间 O(n+m)；简单图常写 O((n+m)log(n+1))。最短路树：run 后每个 v≠s 的 pre 入边构成一棵最短路树；要总边权最小，就在满足 dis[u]+w=dis[v] 的入边中选 w 最小者。Johnson 全源最短路：加超级源以 0 权连所有点，用 BellmanFord 求势 h，把边权改成 w+h[u]-h[v]≥0 后对每个起点跑 Dijkstra，真实距离为 d+h[t]-h[s]，O(nm log m)。

```
struct Dijkstra
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<ll> dis;
    vector<int> pre;

    Dijkstra(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w)
    {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        dis[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u])
                if (w < inf - d && d + w < dis[v])
                {
                    dis[v] = d + w;
                    pre[v] = u;
                    q.push({dis[v], v});
                }
        }
    }
```

```
39
40      vector<int> path(int t) const
41      {
42          if (dis[t] == inf) return {};
43          vector<int> ans;
44          for (; t != -1; t = pre[t]) ans.push_back(t);
45          reverse(ans.begin(), ans.end());
46          return ans;
47      }
48  };
```

模板题使用：Luogu P4779

题意：给非负权有向图和源点 s，输出 s 到各点的最短距离。加完边后 run(s)，读取 dis[1..n]；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 45 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4779

```
1  int main()
2  {
3      std::ios::sync_with_stdio(false);
4      std::cin.tie(nullptr);
5      int n, m, s;
6      std::cin >> n >> m >> s;
7      Dijkstra graph(n);
8      while (m--)
9      {
10         int u, v;
11         long long w;
12         std::cin >> u >> v >> w;
13         graph.add(u, v, w);
14     }
15     graph.run(s);
16     for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
17 }
```

3.1.2 Bellman–Ford：负环影响与路径方案

BellmanFord(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 加有向边并返回 0-based 原边号，允许重边、自环和完整 signed64 边权，总边数 ≤INT_MAX。run(s) 返回是否不存在从 s 可达的负环；run(0) 把全体点初值设 0，等价于超级源点，用于查全图负环。每次 run 重置状态；add 后旧结果失效，须重新 run。

运行后 dis[v]==inf 表示不可达；neg[v] 为真表示从起点可达的负环能到达 v，距离为负无穷（这时不可读取 dis 作为答案）；其他点的 dis 为有限 int128 最短距离。path(v) 返回有限最短路的正向原边号；不可达、负无穷或空路径都返回空 vector，须结合 dis/neg 判定。cycle 为空表示无相关负环，否则是一个相关负环的正向原边号，可通过 e[id].u/v/w 还原顶点和权值。run(0) 的有限距离是从任意起点出发的最小值，允许零边路径，不是某个固定单源距离。

O(nm+n+m) 时间、O(n+m) 空间；path 输出 O(路径长)。inf=2^126，全过程 int128 累加，不能无条件转为 long long。在给定 n,m 的 int 边界内，n 轮最多 nm 次更新，标签对应游走至多 nm 条边，绝对值 <2^125，不碰 inf 也不溢出。e/g/dis/pre/neg/cycle 和 ready 只读，不手改。

```
1  struct BellmanFord
2  {
```

```
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<long long> dis;
    vector<int> pre, neg, cycle; // neg[v]: distance is -inf; cycle: edge ids of one negative cycle

    BellmanFord(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts every vertex at 0 to find any negative cycle. True iff none found.
    bool run(int s)
    {
        dis.assign(n + 1, s ? inf : 0);
        pre.assign(n + 1, -1);
        neg.assign(n + 1, 0);
        cycle.clear();
        dis[s] = 0;
        int last = -1;
        for (int step = 0; step < n; step++)
        {
            last = -1;
            for (int id = 0; id < (int)e.size(); id++)
            {
                auto [u, v, w] = e[id];
                if (dis[u] == inf || dis[u] + w >= dis[v]) continue;
                dis[v] = dis[u] + w, pre[v] = id, last = v;
                if (step == n - 1) neg[v] = 1;
            }
            if (last == -1) break;
        }
        if (last != -1)
        {
            int x = last;
            for (int i = 0; i < n; i++) x = e[pre[x]].u;
            for (int y = x; cycle.empty() || y != x; y = e[pre[y]].u) cycle.push_back(pre[y]);
            reverse(cycle.begin(), cycle.end());
        }
        for (bool changed = true; changed;) // everything reachable from round n is -inf
        {
            changed = false;
            for (auto [u, v, w] : e)
                if (neg[u] && !neg[v]) neg[v] = 1, changed = true;
        }
        return cycle.empty();
    }

```

```
    // Finite shortest path as edge ids; empty for unreachable, -inf or t=s.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf || neg[t]) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_B / 含负边的单源最短路

题意：0-based 有向图，n≤1000、m≤2000、边权绝对值 ≤10000。从 r 可达负环时输出 NEGATIVE CYCLE，否则逐点输出距离，不可达输出 INF。转成模板 1-based 编号；本题有限距离在 signed64 内，故可显式转 long long 输出。不可达位置的负环不影响此题判断。

所需模板：BellmanFord（第 46 页，3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_B

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run(s + 1)) cout << "NEGATIVE CYCLE\n";
    else
        for (int u = 1; u <= n; u++)
        {
            if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
            else cout << (long long)t.dis[u] << '\n';
        }
}
```

3.1.3 SPFA：队列松弛、负环判定与有限路径

Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128

与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<long long> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        g[u].push_back(int(e.size()) - 1);
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts from every vertex (difference constraints). False on a negative cycle.
    bool run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s) dis[u] = 0, in[u] = 1, q.push(u);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                int v = e[id].v;
                if (dis[u] + e[id].w >= dis[v]) continue;
                dis[v] = dis[u] + e[id].w, pre[v] = id, len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v]) in[v] = 1, q.push(v);
            }
        }
        return true;
    }

    // Edge ids of the shortest path to t; empty if unreachable or t is a start.
```

```
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 48 页，3.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

3.1.4 Floyd 全源最短路与原边方案

Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 $<2n \cdot 2\hat{}63<2\hat{}95$，int128 与 inf=2^126 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```cpp
struct Floyd
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<long long>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based. False on a negative cycle; then every result is invalid.
    bool run()
    {
        dis.assign(n + 1, vector<long long>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v]) dis[u][v] = w, nxt[u][v] = id;
        }
        for (int k = 1; k <= n; k++)
        {
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
                if (dis[u][k] != inf)
                    for (int v = 1; v <= n; v++)
                        if (dis[k][v] != inf && dis[u][k] + dis[k][v] < dis[u][v])
                            dis[u][v] = dis[u][k] + dis[k][v], nxt[u][v] = nxt[u][k];
        }
        return true;
    }

    // Edge ids from s to t; empty if unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        for (; s != t; s = e[ids.back()].v) ids.push_back(nxt[s][t]);
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 50 页，3.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

3.1.5 杭州 2023 G：开放时间的单位边最短路

QOJ8236。初始蛇身从头到尾编号 $0,\dots,k-1$，第 $i\ge 1$ 节所在点开放时间为 $k-i$（尾格为 1），其余为零；该限制必要且充分。转移 $d_v=\max(d_u+1,\mathrm{open}_v)$。按距离递增处理时首次发现即最优：候选等于 d_u+1 的入 FIFO，其余入开放时间桶，每次取两者较小者；桶指针只扫到下一个 FIFO 距离。$R=\max \mathrm{open}$，时间 $O(V+E+R)$，空间 $O(V+R)$。仅适用于所有边权为 1；一般非负权用优先队列。答案用 unsigned long long 累加距离平方（模 2^{64}）。

3.2 k 短路

3.2.1 非负权严格次短行走

StrictSecondShortest(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，无向边添加两次。允许重边、自环及零权，0<=w<LLONG_MAX。run(s) 返回 dis[v][0/1]，分别为最短距离与严格大于它的最小距离；同长路线不重复计数，允许重复点边，s 到自身包含空路。不存在或长度 >=LLONG_MAX 返回 inf=LLONG_MAX；只保证此阈值以下的精确答案，超界扩展跳过。只读 g/n；可重复求解、加边后重算。O((n+m) log(n+m+1)) 时间，O(n+m) 空间，不返回行走方案。

```
struct StrictSecondShortest
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    StrictSecondShortest(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<array<ll, 2>> run(int s) const
    {
        assert(1 <= s && s <= n);
        vector<array<ll, 2>> dis(n + 1, {inf, inf});
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        dis[s][0] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d > dis[u][1]) continue;
            for (auto [v, w] : g[u])
            {
                if (w >= inf - d) continue;
                ll next = d + w;
                if (next < dis[v][0])
                {
                    swap(next, dis[v][0]);
                    q.push({dis[v][0], v});
                }
                if (dis[v][0] < next && next < dis[v][1])
                {
                    dis[v][1] = next;
                    q.push({next, v});
                }
            }
        }
        return dis;
    }
};
```

3.2.2 正权前 K 短行走

KShortestWalks(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，要求 0<w<LLONG_MAX，零权边/环不在接口范围内。允许自环与重边，每次 add 视为不同边。run(s,t,k) 按非降序返回至多 k 个行走长度，k>=0；不同边序列分别计数，即使等长。允许重复点边，不是简单路径算法。s=t 时第一个答案为空路 0；若排除空路，调用 k+1 后删除首项，需 k<INT_MAX。终点仍可继续走再回来。长度 >=LLONG_MAX 不返回；只有阈值以下答案有保证。每个顶点至多扩展 k 次，O(n+km log(km+2)) 时间、O(n+km) 空间；k 大时应改用更高效算法。只读 g/n，可重复求解和加边后重算，不返回方案。

```
struct KShortestWalks
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    KShortestWalks(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 < w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run(int s, int t, int k) const
    {
        assert(1 <= s && s <= n && 1 <= t && t <= n && k >= 0);
        vector<ll> answer;
        if (k == 0) return answer;
        vector<int> count(n + 1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (count[u] == k) continue;
            count[u]++;
            if (u == t)
            {
                answer.push_back(d);
                if (int(answer.size()) == k) break;
            }
            for (auto [v, w] : g[u])
            {
                if (count[v] < k && w < inf - d)
                {
                    q.push({d + w, v});
                }
            }
        }
        return answer;
    }
};
```

第四章　生成树问题

4.1　最小生成树

4.1.1　Kruskal 重构森林与阈值连通块

代码依赖：dsu（第 194 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

KruskalTree(n), 0≤n≤INT_MAX/2, 原点 0..n-1; add(u,v,w) 加入无向边, 返回插入编号, 边数 ≤INT_MAX, 支持完整 signed64 权、重边、自环、负权和断连。build(descending=false) 重建并返回连通块数；默认升序，true 降序。每次成功合并创建父点：ch 为左右儿子，val 为合并边权；叶点 ch={-1,-1} 且 val 无意义。总点数 k=2n-连通块数，父编号大于子编号；可按 n..k-1 聚合叶点信息，不需要模拟 DFS。up[0] 为父点，根为-1。

component(u,w,strict=false) 返回阈值子图中 u 所属分量的最高重构节点；默认升序保留边权 ≤w，降序保留 ≥w, strict 改成 < 或 >。其子树叶点恰为分量，可从聚合数组查大小、最小值等。同权边必须连续爬完所有合法祖先；不以叶点 val 充当无穷哨兵。构建 O(m log(m+1)+n log(n+1))，空间 O(m+n log(n+1))，查询 O(log(n+1))。依赖 dsu，合并前按大小交换根。支持重复 build、切换方向与值复制；加边后必须重建再查询，构建后成员只读。空图只允许 build、不允许查询。

```
struct KruskalTree
{
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    bool down = false;
    vector<Edge> e;
    vector<long long> val;
    vector<array<int, 2>> ch;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX / 2);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

重建森林与倍增表（接上页同一结构体）：

```
    int build(bool descending = false)
    {
        down = descending;
        ch.assign(n, {-1, -1});
        val.assign(n, 0);
        dsu d(n);
        vector<int> top(n), ids(e.size());
        iota(top.begin(), top.end(), 0);
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(), ids.end(), [&](int a, int b)
        {
            if (e[a].w != e[b].w)
                return down ? e[a].w > e[b].w : e[a].w < e[b].w;
            return a < b;
        });
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            int x = ch.size();
            ch.push_back({top[u], top[v]});
            val.push_back(w);
            d.merge(u, v);
            top[u] = x;
        }
        int k = ch.size();
        int h = max(1, (int)bit_width((unsigned)k));
        up.assign(h, vector<int>(k, -1));
        for (int u = n; u < k; u++)
        {
            for (int v : ch[u]) up[0][v] = u;
        }
        for (int j = 1; j < h; j++)
        {
            for (int u = 0; u < k; u++)
            {
                int v = up[j - 1][u];
                if (v != -1) up[j][u] = up[j - 1][v];
            }
        }
        return 2 * n - k;
    }
```

阈值分量查询（接上页同一结构体）：

```
    int component(int u, long long w, bool strict = false) const
    {
        assert(0 <= u && u < n && !up.empty());
        for (int j = (int)up.size() - 1; j >= 0; j--)
        {
            int v = up[j][u];
            if (v == -1) continue;
            bool ok = down ? val[v] >= w : val[v] <= w;
            if (strict && val[v] == w) ok = false;
            if (ok) u = v;
        }
        return u;
    }
};
```

为何阈值查询正确 升序处理完所有权不超过 w 的边后，并查集恰好是该阈值子图的连通分量。每次成功合并的两棵子树叶点集合不交，新父点记录其并集；沿叶点到根的内部点权单调不减。因此最高合法祖先的叶点集合就是答案。降序情形反向比较；严格阈值排除等权合并。同权内部点可能连续出现，倍增必须爬到最高合法点。

如何聚合分量 叶点先存原数据，随后按编号 n 到 k-1 合并两个儿子的结果，可求大小、最小值、最大值等。不必为聚合另跑 DFS；这利用的是建点时已经确定的父子拓扑顺序。断连图的各棵树独立，无须添加超级根。

归程建模 先用长度求每个点到 1 号点的最短步行距离。汽车只能走海拔严格大于水位的边，故选降序构建和 strict=true；可在分量内任意点下车，答案为该重构子树的叶点距离最小值。普通最小生成树的总权不能代替这个分量查询。

4.1.2 曼哈顿最小生成树

代码依赖：dsu（第 194 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 194 页，D.1.1 节）、ManhattanMST（第 60 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

4.1.3 严格与非严格次小生成树、换边方案

代码依赖：dsu（第 194 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

SecondMST(n)，n≥1，点号 0..n-1。add(u,v,w) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及 y 根挂 x 根；不用按秩合并。时间 O((n+m)log(n+m))，空间 O(m+n log n)，DFS 递归深度 O(n)。merge/dfs/path 与工作数组为内部实现；原始 e 只读。

```
struct SecondMST
{
    using I = __int128_t;
    using Pair = array<int, 2>;

    struct Edge
    {
        int u, v;
        long long w;
    };

    struct Result
    {
        bool connected = false;
        I weight = 0;
        vector<int> tree;
        optional<I> next;
        int in = -1, out = -1;
    };

    int n, h;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Pair>> mx;

    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

不同权摘要与递归预处理（接上页同一结构体）：

```
    // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
    Pair merge(Pair a, Pair b) const
    {
        for (int x : b)
        {
            if (x == -1) continue;
            if (a[0] == -1 || e[x].w > e[a[0]].w)
            {
                a[1] = a[0];
                a[0] = x;
            }
            else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
                a[1] = x;
        }
        return a;
    }

    void dfs(int u, int p)
    {
        for (int j = 1; j < h; j++)
        {
            int v = up[j - 1][u];
            up[j][u] = up[j - 1][v];
            mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
        }
        for (auto [v, id] : g[u])
            if (v != p)
            {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v] = {id, -1};
                dfs(v, u);
            }
    }
```

路径查询（接上页同一结构体）：

```
    Pair path(int u, int v) const
    {
        Pair ans{-1, -1};
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = h - 1; j >= 0; j--)
            if ((dep[u] - dep[v]) >> j & 1)
            {
                ans = merge(ans, mx[j][u]);
                u = up[j][u];
            }
        if (u == v) return ans;
        for (int j = h - 1; j >= 0; j--)
            if (up[j][u] != up[j][v])
            {
                ans = merge(ans, merge(mx[j][u], mx[j][v]));
                u = up[j][u];
                v = up[j][v];
            }
        return merge(ans, merge(mx[0][u], mx[0][v]));
    }
```

Kruskal 与逐边换入（接上页同一结构体）：

```
    Result solve(bool strict = true)
    {
        vector<int> ids(e.size());
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(),
             ids.end(),
             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
        dsu ds(n);
        vector<bool> used(e.size());
        g.assign(n, {});
        Result ans;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (!ds.merge(u, v)) continue;
            used[id] = true;
            ans.weight += (I)w;
            ans.tree.push_back(id);
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        if ((int)ans.tree.size() != n - 1) return {};
        ans.connected = true;
        dep.assign(n, 0);
        up.assign(h, vector<int>(n));
        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
        dfs(0, -1);
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (used[id] || u == v) continue;
            auto p = path(u, v);
            int out = p[0];
            if (strict && e[out].w == w) out = p[1];
            if (out == -1) continue;
            I cost = ans.weight + (I)w - (I)e[out].w;
            if (!ans.next || cost < *ans.next)
            {
                ans.next = cost;
                ans.in = id;
                ans.out = out;
            }
        }
        return ans;
    }
};
```

4.1.4 Kruskal 最小生成树与生成森林

代码依赖：dsu（第 194 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<int> ids;
    long long weight = 0;

    Kruskal(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // Minimum spanning forest on 0..n-1; ties by edge id. Returns the component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        stable_sort(order.begin(), order.end(), [&](int a, int b) { return e[a].w < e[b].w; });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
            if (d.merge(e[id].u, e[id].v))
            {
                ids.push_back(id);
                weight += e[id].w;
                cnt--;
            }
        return cnt;
    }
};
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 194 页，D.1.1 节）、Kruskal（第 67 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

4.1.5 稠密图 Prim 最小生成森林

Prim(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 自动加无向边，允许完整 signed64 权、零权、负权、自环，重边只保留最小权。不保存原边编号；cost 为 int128 矩阵，inf=2^126 表示缺边，−1 和 LLONG_MAX 均为合法边权。

run() 每次重建最小生成森林，返回原图连通块数。n>0 时返回 1 表示 MST，空图返回 0。weight 为 int128 总权；pre[v] 为父点，根为 −1，非根选边权为 cost[pre[v]][v]。父点森林无环，支持断连和孤立点；同权任选，不承诺字典序。加边后须重新 run，支持重复运行与独立复制；成员只读。

构造和存储 $O(n^2)$，add 为 O(1)，run 为 $O(n^2)$，矩阵约 $16n^2$ 字节，按实际内存选择 n；稀疏图优先 Kruskal。每步选到已选点集的最轻边；所有剩余点均无跨边时开启新树，不把 inf 计入总权。只向未选点严格更新父点，负自环不入树。总权绝对值 <2^94，可为 −1 或超出 long long，不能用权值判断断连。

```
struct Prim
{
    static constexpr long long inf = LLONG_MAX / 4;
    int n;
    vector<vector<long long>> cost;
    vector<int> pre;
    long long weight = 0;

    Prim(int n) : n(n), cost(n, vector<long long>(n, inf)) {}

    void add(int u, int v, long long w) { cost[u][v] = cost[v][u] = min(cost[u][v], w); }

    // O(n^2) minimum spanning forest on 0..n-1; returns the component count.
    int run()
    {
        vector<long long> d(n, inf);
        vector<bool> vis(n);
        pre.assign(n, -1);
        weight = 0;
        int cnt = 0;
        for (int step = 0; step < n; step++)
        {
            int u = -1;
            for (int v = 0; v < n; v++)
                if (!vis[v] && (u == -1 || d[v] < d[u])) u = v;
            if (d[u] == inf)
                cnt++;
            else
                weight += d[u];
            vis[u] = true;
            for (int v = 0; v < n; v++)
                if (!vis[v] && cost[u][v] < d[v]) d[v] = cost[u][v], pre[v] = u;
        }
        return cnt;
    }
};
```

模板题使用：AOJ ALDS1_12_A / Minimum Spanning Tree

题意：输入 n 及 n×n 对称邻接矩阵，−1 表示缺边，其余为 0..2000；n≤100 且保证连通。只读取上三角的有效边，输出 MST 总权；本题可安全转 long long。注意输入 −1 缺边是题目约定，Prim.add 的 −1 是合法负权。

所需模板：Prim（第 69 页，4.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Prim g(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int w;
            cin >> w;
            if (u < v && w != -1) g.add(u, v, w);
        }
    g.run();
    cout << (long long)g.weight << '\n';
}
```

4.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 4.1.2 节（第 60 页）。边权 $d(p,q)=|p_x-q_x|+|p_y-q_y|$。每点只保留四个向右闭锥中的最近邻，再对候选边 Kruskal；候选图含至少一棵 MST，但不含所有最优边，禁边等额外约束不能套用。

单方向锥 $q_x-p_x \ge q_y-p_y \ge 0$：按 $(x+y,$原编号$)$ 排序，活动点以 $-y$ 为 map 键，首个满足条件者即最近邻，找到后删除。四轮坐标 $(x,y),(y,x),(-y,x),(x,-y)$。总时间 $O(n\log n)$，空间 $O(n)$。重复坐标是不同顶点，按编号生成零权链。

输入为 signed 64 位，先转 signed 128 位再做和差；单边最大 $2^{65}-2$。返回 0 基编号与 128 位总权；空集与单点返回 0 和空边集。

4.1.7 严格次小生成树与换边

`SecondMST` 先 Kruskal 求 MST，倍增维护树路径上两个不同的最大边权及边号。每条非树非自环边换入：非严格删路径最大边；严格且换入边等于最大值时删第二大不同权边，无则跳过。`tree` 删 `out`、加 `in` 得 `next`；无次小树时 `next` 为空，断图时 `connected=false`。总权用 int128，边号 -1 表示缺项。

4.2 最小树形图

4.2.1 有向最小树形图

solve(n,root,edges)，点号 0..n-1，n≥1、0≤root<n；返回以 root 为根向外可达的最小树形图费用，无解 nullopt。允许负权、重边和自环（自环不选），n=1 的费用为 0；费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^120，类型为 int128。每个非根点选最小入边；若成环则缩点，跨环入边减去原终点已选入边费用，以计算替换代价。O(VE)，edges 按值传入，原图保持不变。仅返回费用，不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验，暂无专门 OJ AC。

```
struct Arborescence
{
    using I = __int128_t;

    struct Edge
    {
        int u, v;
        I w;
    };

    // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
    static optional<I> solve(int n, int root, vector<Edge> e)
    {
        const I inf = I(1) << 120;
        I ans = 0;
        while (true)
        {
            vector<I> in(n, inf);
            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
            for (auto a : e)
                if (a.u != a.v && a.w < in[a.v])
```

```
            {
                in[a.v] = a.w;
                pre[a.v] = a.u;
            }
        in[root] = 0;
        for (I x : in)
        {
            if (x == inf) return nullopt;
            ans += x;
        }
```

判环与缩点（接上页同一函数）：

```
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                int u = i;
                while (vis[u] != i && id[u] == -1 && u != root)
                {
                    vis[u] = i;
                    u = pre[u];
                }
                if (u != root && id[u] == -1)
                {
                    id[u] = cnt;
                    for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
                    ++cnt;
                }
            }
            if (!cnt) return ans;
            for (int i = 0; i < n; i++)
                if (id[i] == -1) id[i] = cnt++;
            for (auto &a : e)
            {
                int v = a.v;
                a.u = id[a.u];
                a.v = id[a.v];
                if (a.u != a.v) a.w -= in[v];
            }
            root = id[root];
            n = cnt;
        }
    }
};
```

模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至 10^4、正权至 10^6；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 71 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4716

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Arborescence::Edge> e;
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({u - 1, v - 1, w});
    }
    auto ans = Arborescence::solve(n, r - 1, e);
    // This problem's answer fits in long long.
    cout << (ans ? (long long)*ans : -1) << '\n';
}
```

第五章　斯坦纳树

5.1　最小斯坦纳树与原边方案

SteinerTree(n)，点号 0..n-1；add(u,v,w) 添加无向边并返回 0-based 原边编号，允许重边、自环、零权。n≥1，w≥0，所有边权总和严格小于 LLONG_MAX/4。solve(terminals) 自动去重，返回 {cost,edges}，edges 为不重复、升序原边编号；空终端返回 0 及空方案，无法连通返回-1 及空方案。输入图不修改，可重复求解；恢复最优连通子图，不额外保证零权环树化，递归回溯。

令 k 为去重终端数。dp[mask][v] 先枚举互补非空子集在 v 合并，再多源 Dijkstra 移动根。时间 O(3^k n+2^k(m+n)log n)，空间 O(2^k n+n+m)。位掩码约束 k≤20，不代表此规模必然能在时限内完成；P6192 为 n≤100、m≤500、k≤10。负权不适用。当前已通过独立边子集枚举和方案校验，P6192 已 100 分通过，排名未核验。

正确性要点　非负权最优连通子图可删环成树。以 v 为根，分叉处将关键点分成两个非空集合，在同根合并；单分支路径由最短路扩展得到。真子集先处理，逐 mask 闭包即可。回溯合并严格缩小 mask，同 mask 前驱遵循 Dijkstra 的确定次序；严格改善避免零权前驱环。原边去重后的连通方案费用不大于 dp，最优性又排除严格更小，故边权和恰等于 cost。

图存储与加边：

```
struct SteinerTree
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        ll w;
    };
    struct Result
    {
        ll cost;
        vector<int> edges;
    };
    int n;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;

    explicit SteinerTree(int n) : n(n), g(n)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(0 <= w && w < inf - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        return id;
    }
```

状态初始化与同根合并（接上页结构体）：

```
    Result solve(vector<int> terminals) const
    {
        sort(terminals.begin(), terminals.end());
        terminals.erase(unique(terminals.begin(), terminals.end()), terminals.end());
        for (int v : terminals) assert(0 <= v && v < n);
        int k = terminals.size();
        assert(k <= 20);
        if (!k) return {0, {}};
        int count = 1 << k;
        vector<vector<ll>> dp(count, vector<ll>(n, inf));
        vector<vector<int>> split(count, vector<int>(n));
        vector<vector<int>> last(count, vector<int>(n, -1));
        for (int i = 0; i < k; i++) dp[1 << i][terminals[i]] = 0;
        for (int mask = 1; mask < count; mask++)
        {
            for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask)
            {
                int other = mask ^ sub;
                if (sub > other) continue;
                for (int v = 0; v < n; v++)
                {
                    ll value = dp[sub][v] + dp[other][v];
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = sub;
                    }
                }
            }
```

多源最短路（接上页 mask 循环）：

```
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
            for (int v = 0; v < n; v++)
                if (dp[mask][v] < inf) pq.push({dp[mask][v], v});
            while (!pq.empty())
            {
                auto [distance, u] = pq.top();
                pq.pop();
                if (distance != dp[mask][u]) continue;
                for (auto [v, id] : g[u])
                {
                    ll value = distance + edges[id].w;
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = 0;
                        last[mask][v] = id;
                        pq.push({value, v});
                    }
                }
            }
        }
```

最优根与递归方案恢复（接上页 solve 函数）：

```
        int root = min_element(dp.back().begin(), dp.back().end()) - dp.back().begin();
        if (dp.back()[root] == inf) return {-1, {}};
        vector<vector<char>> seen(count, vector<char>(n));
        vector<char> used(edges.size());
        auto trace = [&](auto &&self, int mask, int u) -> void
        {
            if (seen[mask][u]) return;
            seen[mask][u] = 1;
            int sub = split[mask][u];
            if (sub)
            {
                self(self, sub, u);
                self(self, mask ^ sub, u);
            }
            else if (last[mask][u] != -1)
            {
                int id = last[mask][u];
                used[id] = 1;
                self(self, mask, edges[id].u ^ edges[id].v ^ u);
            }
        };
        trace(trace, count - 1, root);
        Result result{dp.back()[root], {}};
        for (int id = 0; id < (int)edges.size(); id++)
            if (used[id]) result.edges.push_back(id);
        return result;
    }
};
```

模板题使用：P6192 最小斯坦纳树

题意：输入无向正权图及关键点，输出连接所有关键点的最小边权和。方案边编号可直接读取 solve 返回值的 edges。

所需模板：SteinerTree（第 75 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    SteinerTree tree(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tree.add(u - 1, v - 1, w);
    }
    vector<int> terminals(k);
    for (int &v : terminals)
    {
        cin >> v;
        v--;
    }
    cout << tree.solve(terminals).cost << '\n';
}
```

第六章　连通性相关

6.1　强连通分量

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

6.1.1　函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT_MAX，实际规模须满足内存。

```
struct FunctionalGraph
{
    using U = unsigned long long;
    int n;
    vector<int> component, position, entry, depth, order;
    vector<vector<int>> cycles, up;

    explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }

    void build(vector<int> to)
    {
        assert(to.size() <= INT_MAX);
        n = to.size();
        vector<int> degree(n);
        for (int v : to)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
        order.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] == 0) order.push_back(u);
        for (int i = 0; i < int(order.size()); i++)
        {
            int v = to[order[i]];
            if (--degree[v] == 0) order.push_back(v);
        }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```
        component.assign(n, -1);
        position.assign(n, -1);
        entry.resize(n);
        depth.assign(n, 0);
        cycles.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] && component[u] == -1)
            {
                vector<int> cycle;
                int v = u, id = cycles.size();
                do
                {
                    component[v] = id;
                    position[v] = cycle.size();
                    entry[v] = v;
                    cycle.push_back(v);
                    v = to[v];
                } while (v != u);
                cycles.push_back(move(cycle));
            }
        for (int i = int(order.size()) - 1; i >= 0; i--)
        {
            int u = order[i], v = to[u];
            component[u] = component[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        up.assign(1, to);
        for (int k = 1; (1ULL << k) <= U(n); k++)
        {
            vector<int> row(n);
            for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
            up.push_back(move(row));
        }
    }
```

跳转与最少步数接口（接上页同一结构体）：

```
    int advance(int u, U k) const
    {
        assert(0 <= u && u < n);
        if (k >= U(depth[u]))
        {
            k -= depth[u];
            u = entry[u];
            const auto &cycle = cycles[component[u]];
            return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
        }
        for (int bit = 0; k; bit++, k >>= 1)
            if (k & 1) u = up[bit][u];
        return u;
    }

    int steps(int u, int v) const
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (component[u] != component[v] || depth[u] < depth[v]) return -1;
        if (depth[v] > 0)
        {
            int d = depth[u] - depth[v];
            return advance(u, d) == v ? d : -1;
        }
        int offset = position[v] - position[entry[u]];
        if (offset < 0) offset += int(cycles[component[u]].size());
        return depth[u] + offset;
    }
};
```

模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从 x 走 k 步后的点，n,q≤200000，0≤k≤10^9。题目 1-based 点号读入减 1、输出加 1；k 用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理 O(n log n)，查询 O(log n)，入环后 O(1)。

所需模板：FunctionalGraph（第 81 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1750

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u;
        unsigned long long k;
        scanf("%d%llu", &u, &k);
        printf("%d\n", graph.advance(u - 1, k) + 1);
    }
}
```

6.1.2 Tarjan 强连通分量与缩点

TarjanSCC(n)，点号 1..n；add(u,v) 加有向边，run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号，跨分量边总从大编号指向小编号，可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表，保留重边；若 DP 需要对不同邻点去重，调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 O(n+m)，递归 DFS 另需 O(n) 调用栈；本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
struct TarjanSCC
{
    int n, timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, bel, st;

    TarjanSCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
    }

    void dfs(int u)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        for (int v : g[u])
```

```
        {
            if (!dfn[v])
            {
                dfs(v);
                low[u] = min(low[u], low[v]);
            }
            else if (!bel[v])
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u])
        {
            ++cnt;
            while (true)
            {
                int v = st.back();
                st.pop_back();
                bel[v] = cnt;
                if (v == u) break;
            }
        }
    }

    // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        bel = dfn;
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u]) dfs(u);
        }
    }

    // Retains parallel edges. Use after run().
    vector<vector<int>> dag() const
    {
        vector<vector<int>> result(cnt + 1);
        for (int u = 1; u <= n; u++)
        {
            for (int v : g[u])
            {
                if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
            }
        }
        return result;
    }
};
```

模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为 0..n-1，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 84 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = g.cnt; i >= 1; i--)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

6.1.3 SCC 模板题与速度榜口径

QOJ906 要求按拓扑序输出强连通分量。第 84 页 `TarjanSCC` 的分量编号为逆拓扑序，按 cnt 到 1 输出；`bel[1..n]` 为 1-based，0-based 输入点号先加一、输出时减一。

6.2 双连通分量

6.2.1 点双与边双：共享 lowlink 核心

BiconnectedCore(n)，点号 1..n；add(u,v) 返回原边的 0-based 编号，run() 从头计算，允许重复运行。

点双结果：blocks 为点双的顶点列表，桥两端成块，孤立点单独成块，割点可出现在多个块；cut[u] 标记割点。

边双结果：bridge[id] 标记桥，bel[u] 是 1..cnt 的边双编号；cnt 是全图边双个数。重边按边号处理；自环保留边号但不进 DFS，不是桥，不额外产生点双。一次共享 DFS 后递归染色，O(n+m) 时间和空间，DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林；g、edges 为原图，不在 DFS 期间修改。

```
struct BiconnectedCore
{
    int n, timer = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks;
    vector<int> dfn, low, cut, bridge, bel, st;

    BiconnectedCore(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        if (u != v)
        {
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        return id;
    }
```

```
void dfs(int u, int parent_edge)
{
    dfn[u] = low[u] = ++timer;
    st.push_back(u);
    int children = 0;
    for (auto [v, id] : g[u])
    {
        if (id == parent_edge) continue;
        if (!dfn[v])
        {
            children++;
            dfs(v, id);
            low[u] = min(low[u], low[v]);
            if (low[v] > dfn[u]) bridge[id] = 1;
            if (low[v] >= dfn[u])
            {
                if (parent_edge != -1 || children > 1) cut[u] = 1;
                blocks.push_back({u});
                while (true)
                {
                    int x = st.back();
                    st.pop_back();
                    blocks.back().push_back(x);
                    if (x == v) break;
                }
            }
        }
        else
            low[u] = min(low[u], dfn[v]);
    }
    if (parent_edge == -1 && children == 0) blocks.push_back({u});
}
```

```
void paint(int u)
{
    bel[u] = cnt;
    for (auto [v, id] : g[u])
    {
        if (!bridge[id] && !bel[v]) paint(v);
    }
}
```

```
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        bel = dfn;
        bridge.assign(edges.size(), 0);
        blocks.clear();
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
                st.pop_back();
            }
        }
        for (int u = 1; u <= n; u++)
        {
            if (bel[u]) continue;
            ++cnt;
            paint(u);
        }
    }
};
```

模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore（第 87 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8435

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    const auto &blocks = graph.blocks;
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

6.2.2 边双缩点森林（保留桥编号）

代码依赖：BiconnectedCore（第 87 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

bridge_component_forest(graph) 接收已 run 的 BiconnectedCore，返回按 1..cnt 编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。O(n+m) 时间、O(n) 输出空间，不修改分解结果。

```
vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
{
    int cnt = graph.cnt;
    const auto &edges = graph.edges;
    const auto &bridge = graph.bridge;
    const auto &bel = graph.bel;
    vector<vector<pair<int, int>>> tree(cnt + 1);
    for (int id = 0; id < (int)edges.size(); id++)
    {
```

```
        if (!bridge[id]) continue;
        auto [u, v] = edges[id];
        u = bel[u];
        v = bel[v];
        tree[u].push_back({v, id});
        tree[v].push_back({u, id});
    }
    return tree;
}
```

6.3 圆方树

6.3.1 静态仙人掌森林最短路

Cactus(n)，0≤n<INT_MAX/2，原点 1..n。add(u,v,w) 加入无向边；输入须为无自环无重边的仙人掌森林（每条边最多属于一个简单环），不检查此结构前提。权非负，所有边权之和 ≤LLONG_MAX/4。build() 递归构造桥/环虚点森林并倍增；distance(u,v) 仅接受原点，断连返回-1，同点返回 0。查询前必须 build；加边后必须重建，支持重复 build 和复制，无动态修改或路径方案接口。编号 n+1..cnt 为环虚点；t 是从根向下的带权邻接表，成员只读。环的顶点到最上方割点距离取较短弧，虚点到该割点权 0；LCA 是虚点时，须用两个儿子在环上的较短弧替换根距离贡献。dfs 与 prepare 均递归，环上沿 par 回走仅枚举边，不模拟 DFS 栈。原图 DFS 及环枚举 O(n+m)，总预处理/空间 O((n+m)log(n+1))，查询 O(log(n+1))，递归深度 O(n)。n=0 允许 build 但无合法查询；不支持负权、非法仙人掌或修改内部数组。

```
struct Cactus
{
    using ll = long long;
    int n, m = 0, cnt, timer = 0, lg;
    vector<vector<tuple<int, int, ll>>> g;
    vector<vector<pair<int, ll>>> t;
    vector<vector<int>> up;
    vector<int> dfn, par, bel, dep, root;
    vector<ll> pref, pos, len, dis;

    Cactus(int n) : n(n), g(n + 1)
    {
        assert(0 <= n && n < INT_MAX / 2);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(u != v && w >= 0);
        g[u].push_back({v, m, w});
        g[v].push_back({u, m, w});
        m++;
    }
```

递归构造环虚点与根距离（接上页同一结构体）：

```
    void dfs(int u, int pe)
    {
        dfn[u] = ++timer;
        for (auto [v, id, w] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                par[v] = u;
                pref[v] = pref[u] + w;
                dfs(v, id);
                if (!bel[v]) t[u].push_back({v, w});
            }
            else if (dfn[v] < dfn[u])
            {
                int b = ++cnt;
                len[b] = pref[u] - pref[v] + w;
                t[v].push_back({b, 0});
                for (int x = u; x != v; x = par[x])
                {
                    bel[x] = b;
                    pos[x] = pref[x] - pref[v];
                    t[b].push_back({x, min(pos[x], len[b] - pos[x])});
                }
            }
        }
    }

    void prepare(int u)
    {
        for (int k = 1; k < lg; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (auto [v, w] : t[u])
        {
            up[0][v] = u;
            dep[v] = dep[u] + 1;
            root[v] = root[u];
            dis[v] = dis[u] + w;
            prepare(v);
        }
    }
```

从零重建（接上页同一结构体）：

```
    void build()
    {
        cnt = n;
        timer = 0;
        int size = 2 * n + 1;
        t.assign(size, {});
        dfn.assign(size, 0);
        par = bel = dep = root = dfn;
        pref.assign(size, 0);
        pos = len = dis = pref;
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) dfs(u, -1);
        lg = 1;
        while ((1LL << lg) <= cnt) lg++;
        up.assign(lg, vector<int>(size));
        for (int u = 1; u <= n; u++)
        {
            if (dfn[u] && !par[u])
            {
                root[u] = u;
                prepare(u);
            }
        }
    }
```

查询与环上距离修正（接上页同一结构体）：

```
    // Original vertices only; disconnected pairs return -1.
    ll distance(int u, int v) const
    {
        if (root[u] != root[v]) return -1;
        ll ans = dis[u] + dis[v];
        if (dep[u] < dep[v]) swap(u, v);
        int delta = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
            if (delta >> k & 1) u = up[k][u];
        if (u == v) return ans - 2 * dis[u];
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        int p = up[0][u];
        if (p <= n) return ans - 2 * dis[p];
        ll arc = abs(pos[u] - pos[v]);
        return ans - dis[u] - dis[v] + min(arc, len[p] - arc);
    }
};
```

为什么需要环上修正 桥和环块构成森林，块间路径唯一。若最近公共祖先是原点，两条根距离减去公共前缀即可；若是环虚点，u、v 现为该虚点的两个原点儿子，先扣掉这两点到根的距离，再加环上两段弧长的较小值。不能直接把虚点当普通 LCA，否则会强迫路径绕经环顶割点。

复杂度与前提 每条边只属于至多一个环，因此沿 par 回走的总次数为 O(m)，不是每个返祖边各走 O(n)。两次 DFS 均为递归。输入须为非负权简单仙人掌森林，全部边权和不超过 LLONG_MAX/4；不自动校验结构，非法图可能越界。加边后必须重建。

模板题使用：P5236 【模板】静态仙人掌

题意：正权无重边仙人掌，查询两点最短距离；n,q≤10000、m≤20000、w≤100000。

所需模板：Cactus（第 91 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5236

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Cactus solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    solver.build();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << solver.distance(u, v) << '\n';
    }
}
```

6.3.2 由点双构造圆方森林

代码依赖：BiconnectedCore（第 87 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

block_cut_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```
vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
{
    int n = graph.n;
    const auto &blocks = graph.blocks;
    vector<vector<int>> tree(n + blocks.size() + 1);
    for (int i = 0; i < (int)blocks.size(); i++)
    {
        int b = n + i + 1;
        for (int u : blocks[i])
        {
            tree[u].push_back(b);
            tree[b].push_back(u);
        }
    }
    return tree;
}
```

第七章　环计数问题

7.1　简单无向图四元环计数

count_four_cycles(n,edges) 返回 long long 计数，n≥0、点 0..n-1、m=edges.size()≤INT_MAX。输入必须是简单无向图，每条边恰好给一次，不允许重边和自环；不自动去重。允许空图、孤立点、断连，输入不变，可重复调用。计四个不同点组成的四边环，不要求诱导子图无弦；K4 有 3 个四元环，旋转和反向不重复计算。按 (度数, 编号) 严格定序，以唯一最大点 u 为起点，统计 u-v-w 中 v,w 均小于 u 的二步路径；第二条路径与前面路径配对。每轮清空实际访问的 cnt。时间 O(n+m$\sqrt{m}$)，空间 O(n+m)，无需排序或递归；不可仅按编号定序后仍声称此复杂度。cnt 只需 int，答案必须 long long；在 m≤INT_MAX 约定下，环数 ≤m(m-1)/2<2^61，不溢出。

```
long long count_four_cycles(int n, const vector<pair<int, int>> &edges)
{
    assert(n >= 0 && edges.size() <= INT_MAX);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    auto less = [&](int u, int v)
    {
        return pair{g[u].size(), u} < pair{g[v].size(), v};
    };
    vector<int> cnt(n), touched;
    long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (!less(v, u)) continue;
            for (int w : g[v])
            {
                if (!less(w, u)) continue;
                if (cnt[w] == 0) touched.push_back(w);
                ans += cnt[w];
                cnt[w]++;
            }
        }
        for (int w : touched) cnt[w] = 0;
        touched.clear();
    }
    return ans;
}
```

模板题使用：U367189 无向图四元环计数

题意：简单无向图，n≤100000，m≤200000；允许断连，不含重边和自环。输入 1-based 端点，输出非诱导四元环数。

所需模板：count_four_cycles（第 97 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U367189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    cout << count_four_cycles(n, edges) << '\n';
}
```

7.2 无向简单图三元环枚举

enumerate_triangles(n,edges,visit) 使用 0..n-1 点号；edges 须为无向简单图，无自环、无重边，每条边只给一次，端点顺序任意。每个三角形恰调用一次 visit(u,v,w)，参数按度数和编号组成的次序排列，不保证编号递增。可在回调中计数、累计权值或收集方案；计数用 long long。按 (度数, 编号) 从小到大定向，标记 u 的出邻点后枚举 u 到 v 到 w 的有向路径。每个出度为 $O(\sqrt{m})$，总时间 $O(n+m\sqrt{m})$，含常数时间回调，空间 O(n+m)；空图合法。调用期间不修改输入边集。

```
template <class F>
void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
{
    assert(n >= 0);
    vector<int> deg(n), mark(n, -1);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        deg[u]++, deg[v]++;
    }
    for (auto [u, v] : edges)
    {
        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
        g[u].push_back(v);
```

```
    }
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u]) mark[v] = u;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w] == u) visit(u, v, w);
    }
}
```

模板题使用：Library Checker enumerate_triangles

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号 0..n-1。

所需模板：enumerate_triangles（第 98 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_triangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) cin >> u >> v;
    const long long mod = 998244353;
    long long ans = 0;
    enumerate_triangles(n,
                        edges,
                        [&](int u, int v, int w)
                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
    cout << ans << '\n';
}
```

第八章　最小环

相关应用：Karp 最小平均权有向环，目标不同于最小总权环；两遍 DP 线性空间。

8.1　有向图最小平均权环：线性空间 Karp

minimum_mean_cycle(n,edges)，0≤n<INT_MAX，点 0..n-1，edges 为 tuple<int,int,T> 有向边；支持断连、负权、自环和重边，输入不变。返回 optional<pair<W,int»，无环 nullopt；有环返回分子/正分母，未约分，不返回环方案。T 为有符号至多 64 位整数时 W=int128，全部比较精确，支持 LLONG_MIN/MAX；n<2^31 使交叉乘法绝对值 <2^126。实数 T 须支持值语义，W=T；禁止 NaN/Inf，要求 $2n^2\max|w|<2^{118}$ 避免溢出，仍有舍入误差，不能泛称精确。P3199 驱动用 GCC ___float128 进行 113 位核心运算，long double 仅用于输入输出；需 GCC 支持此扩展。F0 全部为 0，Fk 为恰好 k 条边的最短行走；先滚动到 Fn，再重算 F0..Fn-1，取 min_v max_k (Fn(v)-Fk(v))/(n-k)。时间 O(n(n+m))，额外 O(n)，输入 O(m)；非强连通也成立，无需 DFS、二分或负环启发式。k 为 0..n-1，不能只比较最后一层或允许至多 k 条边。

```
template <class T>
auto minimum_mean_cycle(int n, const vector<tuple<int, int, T>> &edges)
{
    static_assert(!is_integral_v<T> || (is_signed_v<T> && sizeof(T) <= 8));
    assert(0 <= n && n < INT_MAX);
    using W = conditional_t<is_integral_v<T>, __int128_t, T>;
    using Ratio = pair<W, int>;
    const W inf = W(1LL << 60) * W(1LL << 60);
    vector<W> d(n), next(n);
    auto step = [&]()
    {
        fill(next.begin(), next.end(), inf);
        for (auto [u, v, w] : edges)
        {
            if (d[u] == inf) continue;
            W value = d[u] + w;
            if (value < next[v]) next[v] = value;
        }
        d.swap(next);
    };
    for (int k = 0; k < n; k++) step();
    auto last = d;
    fill(d.begin(), d.end(), 0);
    auto less = [&](const Ratio &a, const Ratio &b)
    {
        return a.first * b.second < b.first * a.second;
    };
```

第二遍重算并取 min-max（接上页同一函数）：

```
    vector<Ratio> best(n, {0, 0});
    for (int k = 0; k < n; k++)
    {
        for (int v = 0; v < n; v++)
        {
            if (last[v] == inf || d[v] == inf) continue;
            Ratio now{last[v] - d[v], n - k};
            if (best[v].second == 0 || less(best[v], now))
                best[v] = now;
        }
        if (k + 1 < n) step();
    }
    optional<Ratio> ans;
    for (auto r : best)
    {
        if (r.second && (!ans || less(r, *ans))) ans = r;
    }
    return ans;
}
```

为什么全点初始化成立 把各边减去最小平均值后，所有环非负且存在零环。任意起点最短行走都能删环变成少于 n 条边的路径，所以每点的 max 比值非负。到零环的最短行走接无限重复零环，取长度 n 的最短前缀，其终点使 max 为 0；因此全局 min 为 0。恢复原权后所有比值增加同一常数，公式仍成立。无环图没有长度 n 行走，返回 nullopt。

返回分数与数值类型 分母为 1..n；不保证约分，也不保证分子本身就是某个环的权和。整数类型提升到 int128 进行运算和交叉比较；小数用调用者选择的 T，普通浮点不是精确计算。P3199 采用 GCC 的 113 位浮点进行核心运算，long double 仅用于输入输出。无需保存全部 DP 层，但第二遍必须重新从全 0 开始。

模板题使用：P3199 [HNOI2009] 最小圈

题意：实数有向边权，n≤3000、m≤10000、|w|≤10^7，保证有圈；GCC ___float128 核心运算，long double 读入/输出 8 位小数。

所需模板：minimum_mean_cycle（第 100 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3199

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Real = __float128;
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, Real>> edges;
    while (m--)
    {
        int u, v;
        long double w;
        cin >> u >> v >> w;
        edges.emplace_back(u - 1, v - 1, w);
    }
    auto ans = minimum_mean_cycle(n, edges);
    assert(ans);
    cout << fixed << setprecision(8) << (long double)(ans->first / ans->second) << '\n';
}
```

8.2 非负权无向图最小环与原边方案

MinCycle(n)，n≥0，点 0..n-1。add(u,v,w) 返回从 0 开始的原边号，边数 ≤INT_MAX，0≤w≤LLONG_MAX；允许自环和重边。solve() 从零求至少 3 个不同顶点的简单环，无解 nullopt；故自环忽略，平行边只保留最轻的一条（同权留最早），不计二元环。Result.weight 是 int128，vertices 不重复首点，edges[i] 连接 vertices[i] 与 vertices[(i+1)%size]，边号对应 e；并列返回任意一个，不保证字典序。支持空图、断连、零权、重复 solve 及加边后重算，旧结果独立；e/id 只读。Floyd 更新中间点 k 之前枚举 i<j<k，用 d[i][j]+ 边 (i,k)+ 边 (j,k) 更新最优值，每轮至多恢复一次路径；严格松弛避免零权环导致方案循环。solve 为 $O(n^3+n^2)$，构造 $O(n^2)$，add 摊还 O(1)，总空间 $O(n^2+m)$，恢复通过 next 逐边走路径，不是模拟 DFS。负边不支持；无穷大 2^120 只作内部哨兵，合法简单路径与候选和均小于 2^95。

```
struct MinCycle
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };
    struct Result
    {
        I weight;
        vector<int> vertices, edges;
    };

    int n;
```

```
    vector<Edge> e;
    vector<vector<int>> id;

    MinCycle(int n) : n(n), id(n, vector<int>(n, -1))
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        int k = e.size();
        e.push_back({u, v, w});
        if (u != v && (id[u][v] == -1 || w < e[id[u][v]].w))
            id[u][v] = id[v][u] = k;
        return k;
    }
```

初始化与本轮候选检查（接上页同一结构体）：

```
    optional<Result> solve() const
    {
        const I inf = I(1) << 120;
        vector<vector<I>> d(n, vector<I>(n, inf));
        vector<vector<int>> next(n, vector<int>(n, -1));
        for (int i = 0; i < n; i++)
        {
            d[i][i] = 0;
            for (int j = 0; j < n; j++)
            {
                if (id[i][j] == -1) continue;
                d[i][j] = e[id[i][j]].w;
                next[i][j] = j;
            }
        }
        Result ans{inf, {}, {}};
        for (int k = 0; k < n; k++)
        {
            int u = -1, v = -1;
            for (int i = 0; i < k; i++)
            {
                if (id[i][k] == -1) continue;
                for (int j = i + 1; j < k; j++)
                {
                    if (id[j][k] == -1 || d[i][j] == inf) continue;
                    I w = d[i][j] + e[id[i][k]].w + e[id[j][k]].w;
                    if (w >= ans.weight) continue;
                    ans.weight = w;
                    u = i;
                    v = j;
                }
            }
```

方案恢复与 Floyd 更新（接上页 solve 的 k 循环）：

```
            if (u != -1)
            {
                ans.vertices = {k, u};
                ans.edges = {id[k][u]};
                while (u != v)
                {
                    int x = next[u][v];
                    ans.edges.push_back(id[u][x]);
                    ans.vertices.push_back(x);
                    u = x;
                }
                ans.edges.push_back(id[v][k]);
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (d[i][k] == inf || d[k][j] == inf) continue;
                    I w = d[i][k] + d[k][j];
                    if (w >= d[i][j]) continue;
                    d[i][j] = w;
                    next[i][j] = next[i][k];
                }
            }
        }
        if (ans.weight == inf) return nullopt;
        return ans;
    }
};
```

检查必须在松弛之前 固定环中最大编号点 k，其两侧邻点 i、j 都小于 k。此时 d[i][j] 只允许小于 k 的中间点，与两条原边共同构成合法简单环；先做本轮 Floyd 会使路径经过 k，错误接受重复点。

恢复不能每次改进都做 每轮只记最优 i、j，结束枚举后恢复一次，所有恢复共 $O(n^2)$。next 沿真实路径逐边走，不是模拟 DFS；更新必须严格变小，尤其不能把零权等长路径改成自指。

模板题使用：P6175 无向图的最小环问题

题意：求至少 3 个不同顶点的最小权环；n≤100，m≤5000，边权 1..100000，答案 ≤10000000，可转 long long 输出；无解精确输出 No solution.。

所需模板：MinCycle（第 102 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6175

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    auto ans = g.solve();
    if (ans)
        cout << (long long)ans->weight << '\n';
    else
        cout << "No solution.\n";
}
```

第九章　2-SAT

9.1　2-SAT 与任意可行方案

代码依赖：TarjanSCC（第 84 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

TwoSAT(n)，变量 1..n；add(x,a,y,b) 添加 (x=a) 或 (y=b)，强制值写 add(x,a,x,a)，蕴含 (x=a)=>(y=b) 写 add(x,!a,y,b)。solve() 返回可满足性，仅成功时读取 ans[1..n]；可重复求解和追加子句后重算，空公式可满足。不保证字典序最小。依赖同文件 TarjanSCC：分量编号为逆拓扑序，故真值分量编号较小时取真；换用拓扑序编号的实现须反转不等号。O(n+m) 时间和空间，DFS 递归。

```
struct TwoSAT
{
    int n;
    TarjanSCC g;
    vector<int> ans;

    TwoSAT(int n) : n(n), g(2 * n) {}

    int id(int x, bool value) const { return 2 * x - 1 + value; }

    void add(int x, bool a, int y, bool b)
    {
        g.add(id(x, !a), id(y, b));
        g.add(id(y, !b), id(x, a));
    }

    bool solve()
    {
        g.run();
        ans.assign(n + 1, 0);
        for (int x = 1; x <= n; x++)
        {
            if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
            ans[x] = g.bel[id(x, 1)] < g.bel[id(x, 0)]; // Tarjan ids are reverse topological
        }
        return true;
    }
};
```

模板题使用：Library Checker two_sat

题意：每个子句是两个文字的或，判断是否可满足并输出一组赋值。输入为 DIMACS：首行 p cnf n m，正数表示变量，负数表示取反，每个子句末尾为 0。

所需模板：TarjanSCC（第 84 页，6.1.2 节）、TwoSAT（第 107 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/two_sat

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string p, cnf;
    int n, m;
    cin >> p >> cnf >> n >> m;
    TwoSAT g(n);
    for (int i = 0; i < m; i++)
    {
        int a, b, zero;
        cin >> a >> b >> zero;
        g.add(abs(a), a > 0, abs(b), b > 0);
    }
    if (!g.solve())
    {
        cout << "s UNSATISFIABLE\n";
        return 0;
    }
    cout << "s SATISFIABLE\nv";
    for (int i = 1; i <= n; i++) cout << ' ' << (g.ans[i] ? i : -i);
    cout << " 0\n";
}
```

9.2 2-SAT 编号方向与双连通定义

TwoSAT 的 add(x,a,y,b) 表示 $(x=a)\vee(y=b)$；强制值 add(x,a,x,a)，蕴含 $(x=a)\Rightarrow(y=b)$ 写 add(x,!a,y,b)。Tarjan 逆拓扑编号下真文字分量编号更小时取真；换 SCC 实现须检查不等号。solve 仅成功时读 ans，$O(n+m)$，不保证字典序最小。

点双：连通且无割点；桥两端成块，孤立点单独成块，割点可属多块；自环不产生点双、不标桥。每条无向边只 add 一次；lowlink 只跳过父边编号，不跳过重边。BiconnectedCore 点号 $1\dots n$、边号从零；blocks 为点双顶点集，bridge 为桥标记，bel 为 $1\dots cnt$ 的边双号（不承诺拓扑方向）。不连通时圆方图与桥缩点均为森林。

removal_components 返回原块数 before 与删点后总块数 after；割点即 after[u]>before。连通无向图能强连通定向当且仅当无桥：DFS 树边向下、非树边向祖先。

圆方森林原点 $1\dots n$，第 i 个点双（从零）对应方点 $n+i+1$。原点权 -1、方点权为块大小时，s,f 间圆方树路径权和为可作中间点的顶点数。所在树原点数 C，删 u 后各支原点数 a_i，有序端点对经过 u 的次数为 $2\sum_{i<j}a_ia_j+2(C-1)[u\le n]$；sz 仅计原点，用 long long，总 $O(n+m)$。

桥森林邻接项为（边双号，原桥号）。连通图有 L 个叶子边双时，补成边双连通至少且恰需 $\lceil L/2\rceil$ 条边：DFS 序收集叶子，奇数补首叶，前后半段配对；仅一个边双不加边；非连通森林不适用。

第十章　欧拉图

10.1　有向图字典序最小欧拉路

构造 DirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边编号，支持自环与重边。run() 返回是否存在欧拉路；参数 start 默认 0，lexicographic 默认 true。start=0 自动选择最小可行起点，否则固定起点；孤立点不影响有边部分的连通性。成功后 vertices 长度为 m+1，edge_ids 长度为 m，第 i 条原边从 vertices[i] 指向 vertices[i+1]；失败清空两数组。无边时返回单点路径，自动起点为 1。可重复 run，add 会清空旧方案。默认求顶点序列字典序最小方案；关闭排序后只保证任意方案，邻接顺序可能保留上次排序。开启排序总时间 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)。DFS 递归深度可能达到 m+1，不是 n。度数检查后，以消费边数判定是否遍历全部非孤立部分，逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号，P7771 双风格完整程序本地验证，在线待补。

```
struct DirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> in, out, next, vertices, edge_ids;

    DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
    {
        assert(n > 0);
    }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        out[u]++;
        in[v]++;
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        int source = 0, sink = 0;
        for (int u = 1; u <= n; u++)
        {
            int delta = out[u] - in[u];
            if (delta == 1 && !source)
                source = u;
            else if (delta == -1 && !sink)
                sink = u;
            else if (delta != 0)
                return false;
        }
        if (bool(source) != bool(sink)) return false;
        if (source)
        {
            if (start && start != source) return false;
            start = source;
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (out[u])
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P7771

题意：给有向图，求字典序最小的欧拉路径。run 默认选择可行起点并按字典序遍历；成功后 vertices 含 m+1 个顶点，失败输出 No。点号 1..n。

所需模板：DirectedEuler（第 109 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7771

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    DirectedEuler graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run())
        cout << "No\n";
    else
    {
        for (int u : graph.vertices) cout << u << ' ';
        cout << '\n';
    }
    return 0;
}
```

10.2 无向图欧拉路与边方向

构造 UndirectedEuler(n)，n≥1，点号 1..n；add(u,v) 返回从 0 开始的逻辑边号，自环和重边均保留。run() 返回是否存在欧拉路，参数 start 默认 0（自动），lexicographic 默认 true。自动选最小可行起点；指定起点须满足奇度条件且位于有边部分。成功得到 m+1 个 vertices 和 m 个 edge_ids，失败清空两数组。第 i 条边方向是 vertices[i] 到 vertices[i+1]，与 edges[edge_ids[i]] 比较可判断是否反向，自环任选方向。used 按逻辑边号标记，使两条半边共同只消费一次；自环贡献度数 2。孤立点不影响有边部分；无边图返回指定点或点 1 的单点序列。默认求顶点序列字典序最小方案；关闭排序只保证任意方案，可能保留上次排序。可重复 run，加边清空旧方案。排序开启时 O(n+m log(m+1))，否则 O(n+m)，空间 O(n+m)，DFS 递归深度可能达到 m+1。全小图走法、原边号与方向独立验证，P2731 字典序顶点序列已线上 AC；边编号、失败输入及其他模式仍由本地验证。

```
struct UndirectedEuler
{
    int n;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<int> next, used, vertices, edge_ids;

    UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        vertices.clear();
        edge_ids.clear();
        return id;
    }

    void dfs(int u, int incoming)
    {
        while (next[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][next[u]++];
            if (used[id]) continue;
            used[id] = 1;
            dfs(v, id);
        }
        vertices.push_back(u);
        if (incoming != -1) edge_ids.push_back(incoming);
    }
```

```
    // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
    bool run(int start = 0, bool lexicographic = true)
    {
        assert(0 <= start && start <= n);
        vertices.clear();
        edge_ids.clear();
        vector<int> odd;
        for (int u = 1; u <= n; u++)
            if (g[u].size() % 2) odd.push_back(u);
        if (!odd.empty() && odd.size() != 2) return false;
        if (!odd.empty())
        {
            if (start && start != odd[0] && start != odd[1]) return false;
            if (!start) start = odd[0];
        }
        if (!start)
        {
            start = 1;
            for (int u = 1; u <= n; u++)
                if (!g[u].empty())
                {
                    start = u;
                    break;
                }
        }
        if (lexicographic)
            for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
        fill(next.begin(), next.end(), 0);
        used.assign(edges.size(), 0);
        dfs(start, -1);
        if (edge_ids.size() != edges.size())
        {
            vertices.clear();
            edge_ids.clear();
            return false;
        }
        reverse(vertices.begin(), vertices.end());
        reverse(edge_ids.begin(), edge_ids.end());
        return true;
    }
};
```

模板题使用：Luogu P2731

题意：USACO Training：无向多重图保证有欧拉路，输出字典序最小顶点序列。m≤1024，点号 1..500；直接构造 500 个点，不把孤立点 1 强设为起点。run() 自动选较小奇点，若全偶选最小非孤立点，并按邻点排序。输出 vertices，共 m+1 项；本题不输出 edge_ids，边编号与方向另验。DFS 递归，最深可达 m+1。

所需模板：UndirectedEuler（第 112 页，10.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2731

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    UndirectedEuler graph(500);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    if (!graph.run()) return 0;
    for (int u : graph.vertices) cout << u << '\n';
    return 0;
}
```

10.3 欧拉回路与指定端点定向

代码依赖：Dinic（第 123 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_orientation(n,edges,start,finish) 返回可空的定向边序列。每条输入为 {u,v,type}，type=1 固定 u 到 v，type=0 可反转；点号 1..n，n≥1。省略两个端点或均传 0 时请求任意起点的欧拉回路；两个正端点请求指定起终点的欧拉路径，相等时为指定起点回路。无解返回 nullopt，有解返回与原边顺序对应的端点对；无边图可返回有值空序列。忽略无关孤立点，但正端点须在有边部分。先任意定向可反转边，扣除目标出入度差；每条反转将两端差值分别改变 -2 和 +2，奇数差无解，否则用容量 1 的反转边流调整。检查底层无向连通性后，满流即得到所需方向。最大流图 n+2 点、至多 m+n 条正向边，总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计，空间 O(n+m)。依赖 flow.hpp；返回边交给 DirectedEuler 可生成连续走法，指定端点时保持指定起点。定向不保证字典序最小。

```cpp
// Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
// start=finish=0 requests a circuit; positive endpoints request a specified trail.
optional<vector<pair<int, int>>> mixed_euler_orientation(
    int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
{
    assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
    assert(bool(start) == bool(finish));
    vector<int> parent(n + 1), id;
    vector<long long> degree(n + 1), balance(n + 1);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int u) -> int
    {
        if (parent[u] != u) parent[u] = self(self, parent[u]);
        return parent[u];
    };
    Dinic flow(n + 2);
    vector<pair<int, int>> direction;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(type == 0 || type == 1);
        parent[find(find, u)] = find(find, v);
        degree[u]++;
        degree[v]++;
        balance[u]++;
        balance[v]--;
        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
        direction.push_back({u, v});
    }
```

```
    if (!edges.empty())
    {
        int root = find(find, edges[0][0]);
        for (int u = 1; u <= n; u++)
            if (degree[u] && find(find, u) != root) return nullopt;
        if (start && (!degree[start] || !degree[finish])) return nullopt;
    }
    if (start)
    {
        balance[start]--;
        balance[finish]++;
    }
    int source = n + 1, sink = n + 2;
    long long need = 0;
    for (int u = 1; u <= n; u++)
    {
        if (balance[u] % 2) return nullopt;
        if (balance[u] > 0)
        {
            flow.add(source, u, balance[u] / 2);
            need += balance[u] / 2;
        }
        else if (balance[u] < 0)
            flow.add(u, sink, -balance[u] / 2);
    }
    if (flow.flow(source, sink) != need) return nullopt;
    for (int i = 0; i < (int)edges.size(); i++)
        if (id[i] != -1 && flow.used(id[i]))
            swap(direction[i].first, direction[i].second);
    return direction;
}
```

10.4 任意起终点的混合欧拉路径

代码依赖：mixed_euler_orientation（第 115 页，10.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

mixed_euler_trail(n,edges) 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 DirectedEuler 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```
// Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
// orders.
optional<vector<pair<int, int>>> mixed_euler_trail(int n,
                                                   const vector<array<int, 3>> &edges)
{
    assert(n > 0);
    vector<long long> degree(n + 1);
    vector<int> odd;
    for (auto [u, v, type] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        degree[u]++;
        degree[v]++;
    }
    for (int u = 1; u <= n; u++)
        if (degree[u] % 2) odd.push_back(u);
    if (odd.empty()) return mixed_euler_orientation(n, edges);
    if (odd.size() != 2) return nullopt;
    auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
    if (answer) return answer;
    return mixed_euler_orientation(n, edges, odd[1], odd[0]);
}
```

第十一章　网络流

11.1　最大流

11.1.1　HLPP 最高标号预流推进与流方案

HLPP(n)，2≤n<INT_MAX/2，点 1..n。add(u,v,cap) 返回偶数残量边号 id，允许自环、重边、反向边和零容量；原边数 <INT_MAX/2，全部原容量之和 ≤LLONG_MAX。flow(s,t) 要求 s≠t，每次从零重算并返回总最大流，支持重复调用、换源汇和加边后重算；与 Dinic 返回增量的约定不同。used(id) 读原边净流，min_cut() 返回源侧 0/1 数组（下标 0 不用），只在完整 flow 后有效，加边后须重算。保证非源汇流守恒，多余预流确实退回源点；不止计算最大预流值。e/g 只读，不支持外部改容量或限流。初始反向 BFS 标号，最高标号单链桶、当前弧和 gap；没有周期全局重标号。不可达点初高 n+1，仍须放流。head/next 是活跃点桶链表，不是 DFS 栈；本算法无需 DFS。给出保守 $O(n^2(n+m))$ 时间、O(n+m) 空间界；不据经验或 AC 声称线性。

```
struct HLPP
{
    using ll = long long;
    struct Edge
    {
        int from, to;
        ll cap, initial;
    };
    int n, s, t, top;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> h, count, cur, head, next;
    vector<ll> ex;

    HLPP(int n) : n(n), g(n + 1)
    {
        assert(n >= 2 && n < INT_MAX / 2);
    }

    int add(int u, int v, ll cap)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && cap >= 0);
        int id = e.size();
        e.push_back({u, v, cap, cap});
        e.push_back({v, u, 0, 0});
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
        return id;
    }
```

活跃点、推流与 gap 重标号（接上页同一结构体）：

```
    void activate(int u)
    {
        if (u == s || u == t || !ex[u]) return;
        next[u] = head[h[u]];
        head[h[u]] = u;
        top = max(top, h[u]);
    }

    void push(int id, ll f)
    {
        auto &a = e[id];
        e[id ^ 1].cap += f;
        a.cap -= f;
        ex[a.from] -= f;
        bool empty = ex[a.to] == 0;
        ex[a.to] += f;
        if (empty) activate(a.to);
    }

    void relabel(int u)
    {
        int old = h[u];
        count[old]--;
        h[u] = 2 * n;
        for (int id : g[u])
            if (e[id].cap) h[u] = min(h[u], h[e[id].to] + 1);
        count[h[u]]++;
        cur[u] = 0;
        if (old >= n || count[old]) return;
        for (int v = 1; v <= n; v++)
        {
            if (v != s && h[v] > old && h[v] < n)
            {
                count[h[v]]--;
                h[v] = n + 1;
                count[h[v]]++;
                cur[v] = 0;
            }
        }
        fill(head.begin(), head.end(), -1);
        top = -1;
        for (int v = 1; v <= n; v++)
            if (v != u) activate(v);
    }
```

从零初始化与反向 BFS（接上页同一结构体）：

```
    // Recompute from zero, including after adding edges or changing terminals.
    ll flow(int source, int sink)
    {
        assert(1 <= source && source <= n && 1 <= sink && sink <= n);
        assert(source != sink);
        s = source;
        t = sink;
        for (auto &a : e) a.cap = a.initial;
        h.assign(n + 1, n + 1);
        cur.assign(n + 1, 0);
        ex.assign(n + 1, 0);
        next.assign(n + 1, -1);
        head.assign(2 * n + 1, -1);
        count.assign(2 * n + 1, 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id ^ 1].cap && h[v] == n + 1)
                {
                    h[v] = h[u] + 1;
                    q.push(v);
                }
            }
        }
        h[s] = n;
        for (int u = 1; u <= n; u++) count[h[u]]++;
        top = -1;
        for (int id : g[s])
            if (e[id].to != s && e[id].cap) push(id, e[id].cap);
```

最高标号放流（接上页同一结构体）：

```
        while (top >= 0)
        {
            if (head[top] == -1)
            {
                top--;
                continue;
            }
            int u = head[top];
            head[top] = next[u];
            while (ex[u])
            {
                if (cur[u] == int(g[u].size()))
                {
                    relabel(u);
                    continue;
                }
                int id = g[u][cur[u]];
                if (e[id].cap && h[u] == h[e[id].to] + 1)
                    push(id, min(ex[u], e[id].cap));
                else
                    cur[u]++;
            }
        }
        return ex[t];
    }
```

原边流量与最小割（接上页同一结构体）：

```
    ll used(int id) const
    {
        return e[id].initial - e[id].cap;
    }

    vector<int> min_cut() const
    {
        vector<int> seen(n + 1);
        queue<int> q;
        seen[s] = 1;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
            {
                int v = e[id].to;
                if (e[id].cap && !seen[v])
                {
                    seen[v] = 1;
                    q.push(v);
                }
            }
        }
        return seen;
    }
};
```

预流不能冒充流方案 不可达汇点的活跃点仍须继续放流，直到多余流退回源点；否则虽然 ex[t] 可能已是最大流值，used 仍可能违反中间点流守恒。完成后，每对残量容量之和等于原容量，各原边流量合法；源侧残量可达集给出同值最小割。

桶与 gap head[h] 和 next[u] 是活跃点单链桶，不是 DFS 栈。点仅在多余流从 0 变正时加入，取出后放流至 0；gap 改变多个点高度后重建桶，并排除当前正在处理的点，避免重复入桶或旧高度记录。初始 BFS 只做一次，反向遍历必须看反边残量；没有周期全局重标号。

调用约定 flow 每次清空旧流并重算，因此允许换源汇；不返回增量，也不支持限流。used 只接收 add 返回的偶数边号，min_cut 只在完整求流后有效。容量总和不超过 LLONG_MAX，不能只保证最大流值未溢出而忽略初始化预流。

模板题使用：P4722 【模板】最大流加强版 / 预流推进

题意：n≤1200、m≤120000，有向容量，输出最大流；允许重边/反向边。

所需模板：HLPP（第 118 页，11.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4722

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    HLPP g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

11.1.2 最大流与方案

Dinic(n)，点号 1..n；add(u,v,c) 返回偶数原边编号；flow(s,t,limit) 返回本次新增流；used(id) 查询原边净流，cut(s) 返回残量可达点集。允许重边、自环，容量非负，s≠t；重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$，递归增广深度 O(V)。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流)，change_edge(id,cap,f) 修改总容量与流，要求 id 为 add 返回的偶数原边编号且 0≤f≤cap；正向残量变为 cap-f，反向为 f。仅保证此残量对一致，不自动维持所有顶点的流量守恒；继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法，缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 O(1)，尚只有本地验证。可选容量缩放：按源码各处 Scaling 注释同步修改 BFS、DFS、递归调用和外层循环，不能只取消一处注释。阈值从不超过当前最大残量 mx 的最大二次幂降到 1，mx=0 直接返回；用 lim≤mx/2 再翻倍避免溢出。bfs/dfs 的 lim 默认 1，cut 仍检查全部正残量；flow 仍返回新增流且支持限流续流。缩放一般 O(VE log(U+1))，U 为本次初始最大残量；不保证比普通版快，大 INF 可能增加空阶段。

```
struct Dinic
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll cap, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> dep, cur;

    Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}

```

```
18      int add(int u, int v, ll c)
19      {
20          assert(c >= 0);
21          int id = (int)e.size();
22          e.push_back({u, v, c, c});
23          e.push_back({v, u, 0, 0});
24          g[u].push_back(id);
25          g[v].push_back(id + 1);
26          return id;
27      }
```

```
    bool bfs(int s, int t) // Scaling: add parameter ll lim = 1.
    {
        fill(dep.begin(), dep.end(), -1);
        queue<int> q;
        dep[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int id : g[u])
                // Scaling: replace e[id].cap with e[id].cap >= lim.
                if (e[id].cap && dep[e[id].to] < 0)
                {
                    dep[e[id].to] = dep[u] + 1;
                    q.push(e[id].to);
                }
        }
        return dep[t] >= 0;
    }

    ll dfs(int u, int t, ll f) // Scaling: add parameter ll lim = 1.
    {
        if (u == t) return f;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int id = g[u][i], v = e[id].to;
            // Scaling: replace !e[id].cap with e[id].cap < lim.
            if (!e[id].cap || dep[v] != dep[u] + 1) continue;
            ll d = dfs(v, t, min(f, e[id].cap)); // Scaling: pass lim last.
            if (d)
            {
                e[id].cap -= d;
                e[id ^ 1].cap += d;
                return d;
            }
        }
        return 0;
    }
```

```
    // Returns additional flow; reuse only with the same source/sink.
    ll flow(int s, int t, ll limit = LLONG_MAX)
    {
        assert(s != t && limit >= 0);
        ll ans = 0;
        // Scaling: mx = maximum current residual cap over all e (both directions).
        // Scaling: if mx == 0, return 0.
        // Scaling: start ll lim = 1, double while lim <= mx / 2.
        // Scaling: wrap the whole while below in for (; lim; lim >>= 1).
        while (ans < limit && bfs(s, t)) // Scaling: pass lim last.
        {
            fill(cur.begin(), cur.end(), 0);
            while (ans < limit)
            {
                ll d = dfs(s, t, limit - ans); // Scaling: pass lim last.
                if (!d) break;
                ans += d;
            }
        }
        return ans;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }

    tuple<int, int, ll, ll> get_edge(int id) const
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        return {e[id].from, e[id].to, e[id].initial, used(id)};
    }

    // Changes one residual pair; caller must preserve global flow conservation.
    void change_edge(int id, ll cap, ll f)
    {
        assert(0 <= id && id < (int)e.size() && !(id & 1));
        assert(0 <= f && f <= cap);
        e[id].initial = cap;
        e[id].cap = cap - f;
        e[id ^ 1].initial = 0;
        e[id ^ 1].cap = f;
    }

    vector<int> cut(int s)
    {
        bfs(s, s);
        vector<int> ans;
        for (int u = 1; u <= n; u++)
            if (dep[u] >= 0) ans.push_back(u);
        return ans;
    }
};
```

模板题使用：Luogu P3376

题意：给定有向容量网络与源点、汇点，求最大流。输入点号为 1..n，平行边逐条加入。

所需模板：Dinic（第 123 页，11.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3376

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    Dinic g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        g.add(u, v, c);
    }
    cout << g.flow(s, t) << '\n';
}
```

11.1.3 排列约束与最大流单位边分类

已知元素切开未知段：段内候选来自原排列对应区间，需求为未知位置数；已知元素单独成需求一的组，首尾设虚拟锚点，保留零需求组。源连左组、右组连汇，容量为需求，每个元素是左右组间一条单位边；流量达到 k 等价于存在合法集合。满流后在残量网络求 SCC，判断能否沿环改变某条单位边，得到必选/可选/不可选。本题增广求流 $O(n^2)$，SCC 为 $O(n)$。

11.2 最小割

11.2.1 最大权闭合子图与选点方案

代码依赖：Dinic（第 123 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

maximum_closure(weight,dependencies) 返回 (最大权值, 选中点列表), weight 为 0-based vector<long long>，图中点号 1..n。依赖边 (u,v) 表示选 u 必须选 v，允许环、自环与重边；允许空选集，因此答案不小于 0。依赖 flow.hpp 中的 Dinic。正权连源、负权连汇，依赖边容量取正权总和加 1；要求所有正权之和严格小于 LLONG_MAX。负权先提升到 int128 再取负，并截断到依赖边容量，因此 LLONG_MIN 也可处理。返回残量图源点可达的原顶点，权值为正权总和减最小割；超大负权边不会出现在最优割中，截断不改变原问题最优值。每次调用重建网络。复杂度沿用 Dinic：一般 $O(V^2E)$，空间 O(V+E)，增广保持递归。返回一种最优闭合集，不承诺最大或最小选点个数。

```
pair<long long, vector<int>> maximum_closure(const vector<long long> &weight,
                                             const vector<pair<int, int>> &dependencies)
{
    using I = __int128_t;
    int n = weight.size(), s = n + 1, t = n + 2;
    I positive = 0;
    for (long long w : weight)
        if (w > 0) positive += w;
    assert(positive < LLONG_MAX);
    long long inf = (long long)positive + 1;
    Dinic graph(n + 2);
    for (int u = 1; u <= n; u++)
    {
        long long w = weight[u - 1];
        if (w > 0)
            graph.add(s, u, w);
        else if (w < 0)
            graph.add(u, t, (long long)min(-I(w), I(inf)));
    }
    for (auto [u, v] : dependencies)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        graph.add(u, v, inf);
    }
    long long answer = (long long)positive - graph.flow(s, t);
    vector<int> selected;
    for (int u : graph.cut(s))
        if (u <= n) selected.push_back(u);
    return {answer, selected};
}
```

11.2.2 正容量最小割的可行边与必选边

代码依赖：Dinic（第 123 页，11.1.2 节）、TarjanSCC（第 84 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

mincut_edges(g,s,t) 只读分析 Dinic 已经完整求出的普通 s→t 最大流；不得传入限流提前停止或人为修改过的残量网络。所有原边初始容量必须严格为正，最大流值须可由 long long 表示。返回逐条原边的 {possible,forced}，对应 Dinic 原始偶数边号 id 的结果是 ans[id/2]。只有饱和边可能属于最小割：两端残量 SCC 不同则可行；起点与 s 同 SCC、终点与 t 同 SCC 则必选。支持平行边和自环，源汇不同，点号 1..g.n。O(n+m) 时间与空间，Tarjan 递归栈 O(n)。零容量边没有正流对应的反向残量边，不能套用本函数的同 SCC 简化条件。每次修改图或重新增广后都需重新分析。

```
vector<array<bool, 2>> mincut_edges(const Dinic &g, int s, int t)
{
    TarjanSCC d(g.n);
    for (auto e : g.e)
        if (e.cap) d.add(e.from, e.to);
    d.run();
    vector<array<bool, 2>> ans(g.e.size() / 2);
    for (int id = 0; id < (int)g.e.size(); id += 2)
    {
        auto e = g.e[id];
        assert(e.initial > 0);
        if (e.cap) continue;
        ans[id / 2][0] = d.bel[e.from] != d.bel[e.to];
        ans[id / 2][1] = d.bel[e.from] == d.bel[s] && d.bel[e.to] == d.bel[t];
    }
    return ans;
}
```

11.2.3 Gomory–Hu 最小割树

代码依赖：Dinic（第 123 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

gomory_hu(n,edges) 返回 n-1 条 (u,v, 容量) 树边；n≥1，原图点号 0..n-1，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 LLONG_MAX。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边，n=1 返回空树。运行 n-1 次 Dinic，一般上界 $O(n^3m+n^2)$，额外空间 O(n+m)，DFS 增广保持递归。P4897 输入 n 表示 n+1 个点，驱动传 n+1。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```
vector<tuple<int, int, long long>>
gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
{
    assert(n >= 1);
    vector<int> parent(n);
    vector<long long> value(n);
    for (int s = 1; s < n; s++)
    {
        int t = parent[s];
        Dinic flow(n);
        for (auto [u, v, w] : edges)
            if (u != v)
            {
                flow.add(u + 1, v + 1, w);
                flow.add(v + 1, u + 1, w);
            }
        long long cut = flow.flow(s + 1, t + 1);
        vector<bool> side(n);
        for (int u : flow.cut(s + 1)) side[u - 1] = true;
        for (int v = 1; v < n; v++)
            if (v != s && parent[v] == t && side[v]) parent[v] = s;
        if (side[parent[t]])
        {
            parent[s] = parent[t];
            parent[t] = s;
            value[s] = value[t];
            value[t] = cut;
        }
        else
            value[s] = cut;
    }
    vector<tuple<int, int, long long>> tree;
    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
    return tree;
}
```

模板题使用：Luogu P4897

题意：给无向图，多次询问不同两点的最小割。点号为 0..n，因此建树传 n+1。再预处理所有点对的答案，查询矩阵额外占 O(n^2) 时间和空间。

所需模板：Dinic（第 123 页，11.1.2 节）、gomory_hu（第 130 页，11.2.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4897

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<tuple<int, int, long long>> edges;
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        scanf("%d%d%lld", &u, &v, &w);
        edges.push_back({u, v, w});
    }
    auto tree = gomory_hu(n + 1, edges);
    vector<vector<pair<int, long long>>> g(n + 1);
    for (auto [u, v, w] : tree)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    int q;
    scanf("%d", &q);
    while (q--)
    {
        int u, v;
        scanf("%d%d", &u, &v);
        // Minimum cut = minimum edge on the tree path.
        long long answer = LLONG_MAX;
        auto dfs = [&](auto &&self, int x, int parent, long long value) -> void
        {
            if (x == v) answer = value;
            for (auto [y, w] : g[x])
                if (y != parent) self(self, y, x, min(value, w));
        };
        dfs(dfs, u, -1, LLONG_MAX);
        printf("%lld\n", answer);
    }
    return 0;
}
```

11.2.4 闭合集、树形图与全局割的完整用法

最大权闭合子图：收益为正权点、成本为负权点，依赖边由前者指向后者；maximum_closure 返回所选点号，共享成本只计一次，总额用 long long。

Arborescence::solve 点号与根 0-based，返回 optional 区分无解与负费用；只返回费用，不恢复边。StoerWagner::solve 要求至少两点、非负对称权，side 为一侧 0-based 原顶点。核心费用为 128 位类型。

11.3 费用流

11.3.1 势能最小费用最大流

MinCostFlow(n),点号 1..n;add(u,v,cap,cost) 返回边编号,flow(s,t,limit) 返回新增流量及 128 位费用,used(id) 返回原边流量。允许负费用，初始残量网络必须没有负环；检测到负环抛异常。Bellman-Ford 初始化 O(VE)，每次增广 O(E log V)；增广次数不能简单假设为 O(V)。另要求 $\sum(\text{cap}\times|\text{cost}|)<2^{120}$，防止总费用超过 int128。slope(s,t,limit) 返回从 (0,0) 开始的本次新增流量/费用折点，相邻同斜率段合并，费用为 int128。无增广时只返回起点；每个整数流量的费用可在线性段内插值。重复调用仍用同源汇，返回值不是历史累计值。flow 不存储曲线；slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证，P3381 驱动只输出终点，不能据此验证全部中间折点。

```
struct MinCostFlow
{
    using ll = long long;
    using i128 = __int128_t;

    struct Edge
    {
        int from, to;
        ll cap, cost, initial;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> g;

    MinCostFlow(int n) : n(n), g(n + 1) {}

    int add(int u, int v, ll cap, ll cost)
    {
        assert(cap >= 0 && cost != LLONG_MIN);
        int id = (int)e.size();
        e.push_back({u, v, cap, cost, cap});
        e.push_back({v, u, 0, -cost, 0});
        g[u].push_back(id);
        g[v].push_back(id + 1);
        return id;
    }

    ll used(int id) const { return e[id].initial - e[id].cap; }
```

```
    // Negative costs allowed; initial residual graph must have no negative cycle.
    pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
    {
        assert(s != t && limit >= 0);
        const i128 inf = i128(1) << 120;
        vector<i128> h(n + 1, 0), d(n + 1);
        vector<int> pre(n + 1);
        // Super-source Bellman-Ford: feasible potentials for every vertex.
        for (int it = 0; it < n; it++)
        {
            bool changed = false;
            for (auto &a : e)
                if (a.cap && h[a.to] > h[a.from] + a.cost)
                {
                    h[a.to] = h[a.from] + a.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cycle");
        }
        ll f = 0;
        i128 cost = 0;
        optional<i128> last;
        while (f < limit)
        {
            fill(d.begin(), d.end(), inf);
            d[s] = 0;
            priority_queue<pair<i128, int>,
                           vector<pair<i128, int>>,
                           greater<pair<i128, int>>>
                q;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (du != d[u]) continue;
                for (int id : g[u])
                {
                    auto &a = e[id];
                    i128 nd = du + a.cost + h[u] - h[a.to];
                    if (a.cap && nd < d[a.to])
                    {
                        d[a.to] = nd;
                        pre[a.to] = id;
                        q.push({nd, a.to});
                    }
                }
            }
```

```
            if (d[t] == inf) break;
            for (int u = 1; u <= n; u++)
                if (d[u] != inf) h[u] += d[u];
            ll take = limit - f;
            for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
            for (int u = t; u != s; u = e[pre[u]].from)
            {
                int id = pre[u];
                e[id].cap -= take;
                e[id ^ 1].cap += take;
                cost += i128(take) * e[id].cost;
            }
            f += take;
            if (curve)
            {
                i128 unit = h[t] - h[s];
                if (last && *last == unit) curve->pop_back();
                curve->push_back({f, cost});
                last = unit;
            }
        }
        return {f, cost};
    }

    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
    {
        return run(s, t, limit, nullptr);
    }

    vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
    {
        vector<pair<ll, i128>> curve{{0, 0}};
        run(s, t, limit, &curve);
        return curve;
    }
};
```

模板题使用：Luogu P3381

题意：给有向容量费用网络，求最大流及对应最小费用。点号 1..n，add 参数依次为起点、终点、容量、单位费用。题目保证费用答案不超过 2^31-1，因此这里可将返回的 int128 转为 long long；一般题目不能照搬转换。

所需模板：MinCostFlow（第 132 页，11.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3381

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow graph(n);
    while (m--)
    {
        int u, v;
        long long capacity, cost;
        cin >> u >> v >> capacity >> cost;
        graph.add(u, v, capacity, cost);
    }
    auto [flow, cost] = graph.flow(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.2 允许负环的最小费用最大流

代码依赖：MinCostFlow（第 132 页，11.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

NegativeCostFlow(n)，点号 1..n；add(u,v,cap,cost) 返回从 0 开始的原边编号。容量非负，所有原边容量总和 ≤LLONG_MAX；费用为 long long 且不等于 LLONG_MIN。允许重边、自环及任意位置的负环。solve(s, t, limit) 只调用一次，limit 默认 LLONG_MAX，s≠t、limit≥0，返回 (不超过 limit 的最大流量, 该流量下的最小费用)，费用为 int128。limit=0 可求最小费用循环流；不足 limit 时返回实际最大流。成功后 used(id) 返回原边流量，自动还原负边退流。先满流负费用边并建立反向正费用退流边，再用超级源汇修复平衡；初始残量费用非负，修复后无负环。删除辅助边后继续最短增广，包含不经过源汇的负环收益。依赖 MinCostFlow，使用两次势能预处理；若总增广次数为 K，时间 O(VE + K(E log(E+1) + V))，空间 O(V+E)，整数容量下为伪多项式，不是费用缩放算法。节点和残量边数须可由 int 表示。

```
struct NegativeCostFlow
{
    using ll = long long;
    using I = __int128_t;
    int n;
    MinCostFlow g;
    vector<ll> balance, cap;
    vector<int> ids;
    vector<bool> reversed;
    ll total = 0;
    I cost = 0;
    bool solved = false;

    NegativeCostFlow(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll c, ll w)
    {
        assert(!solved && 1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= c && c <= LLONG_MAX - total && w != LLONG_MIN);
        total += c;
        int id = ids.size();
        cap.push_back(c);
        reversed.push_back(w < 0);
        if (w < 0)
        {
            balance[u] -= c;
            balance[v] += c;
            cost += I(c) * w;
            ids.push_back(g.add(v, u, c, -w));
        }
        else
            ids.push_back(g.add(u, v, c, w));
        return id;
    }
```

需求修复、增广与原边流量还原（接上页同一结构体）：

```
    pair<ll, I> solve(int s, int t, ll limit = LLONG_MAX)
    {
        assert(!solved && 1 <= s && s <= n && 1 <= t && t <= n && s != t);
        assert(limit >= 0);
        solved = true;
        int first = g.e.size(), back = g.add(t, s, min(total, limit), 0);
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u], 0);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u], 0);
        auto [sent, correction] = g.flow(n + 1, n + 2, need);
        assert(sent == need);
        ll base = g.used(back);
        cost += correction;
        for (int id = first; id < (int)g.e.size(); id++) g.e[id].cap = 0;
        auto [more, extra] = g.flow(s, t, limit - base);
        cost += extra;
        return {base + more, cost};
    }

    ll used(int id) const
    {
        assert(solved && 0 <= id && id < (int)ids.size());
        ll f = g.used(ids[id]);
        return reversed[id] ? cap[id] - f : f;
    }
};
```

模板题使用：Luogu P7173

题意：求允许负费用环的最小费用最大流，独立于源汇的负环也计入费用。无额外流量上限时调用 solve(s,t)。题目容量和费用界保证最终费用可转为 long long；一般模板返回 int128，不能任意照搬此转换。

所需模板：MinCostFlow（第 132 页，11.3.1 节）、NegativeCostFlow（第 136 页，11.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P7173

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    NegativeCostFlow g(n);
    while (m--)
    {
        int u, v;
        long long cap, cost;
        cin >> u >> v >> cap >> cost;
        g.add(u, v, cap, cost);
    }
    auto [flow, cost] = g.solve(s, t);
    cout << flow << ' ' << (long long)cost << '\n';
}
```

11.3.3 SPFA 费用流与残量方案

SPFA 版允许负边，**不允许负环**：每次调用先全点零距离入队检查（含不可达负环），发现即抛异常。最终方案按原边编号查询 used；flow 返回本次新增流量与费用，重复调用须同源汇。容量 long long，距离与费用 int128；费用不能为 LLONG_MIN，总流不超过 long long，且 $\sum cap\,|cost| < 2^{120}$。增广 A 次为 $O((A+1)VE)$；通常优先势能 Dijkstra。

11.4 上下界网络流

11.4.1 上下界可行循环流

代码依赖：Dinic（第 123 页，11.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedCirculation(n)，点号 1..n。add(u,v,lo,hi) 返回原边编号，0≤lo≤hi≤LLONG_MAX，所有下界总和须 ≤LLONG_MAX。solve() 仅调用一次，返回可行性；成功后 used(id) 读取边流量。每点要求流量守恒，属于循环流；带源汇最大流用 BoundedMaxFlow。hi-lo 是残量容量，下界引起 balance；各边上界不必求和，只要求下界总和保证 balance 和超级源总需求不溢出。依赖 Dinic。含负费用环、要求费用最优时用 NegativeCostFlow。

```
struct BoundedCirculation
{
    using ll = long long;
    int n;
    Dinic g;
    vector<ll> balance, lower;
    vector<int> ids;
    bool solved = false, feasible = false;

    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(!solved && 0 <= lo && lo <= hi);
        int id = ids.size();
        ids.push_back(g.add(u, v, hi - lo));
        lower.push_back(lo);
        balance[u] -= lo;
        balance[v] += lo;
        return id;
    }

    bool solve()
    {
        assert(!solved);
        solved = true;
        ll need = 0;
        for (int u = 1; u <= n; u++)
            if (balance[u] > 0)
            {
                g.add(n + 1, u, balance[u]);
                need += balance[u];
            }
            else if (balance[u] < 0)
                g.add(u, n + 2, -balance[u]);
        return feasible = (g.flow(n + 1, n + 2) == need);
    }

    ll used(int id) const
    {
        assert(feasible);
        return lower[id] + g.used(ids[id]);
    }
};
```

模板题使用：LibreOJ 115

题意：无源汇有上下界可行流：每个点流入等于流出，按输入顺序输出每条边的合法流量，无解输出 NO。点号 1..n，n 不超过 200、m 不超过 10200，0≤lo≤hi<3000。add 返回 0 起原始边编号，used(i) 不是 Dinic 残量边号；solve 只能调用一次，且只有可行时才可读取 used。自环、重边逐条保留，不添加源汇回边。

所需模板：Dinic（第 123 页，11.1.2 节）、BoundedCirculation（第 139 页，11.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/115

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    BoundedCirculation graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long lo, hi;
        cin >> u >> v >> lo >> hi;
        graph.add(u, v, lo, hi);
    }
    if (!graph.solve())
    {
        cout << "NO\n";
        return 0;
    }
    cout << "YES\n";
    for (int i = 0; i < m; i++) cout << graph.used(i) << '\n';
}
```

11.4.2 有源汇上下界最值流与边流量

代码依赖：BoundedCirculation（第 139 页，11.4.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

BoundedMaxFlow(n)，点号 1..n；add(u,v,lo,hi) 返回从 0 开始的原边编号，0≤lo≤hi，所有原边上界总和≤LLONG_MAX。solve(s,t) 求最大净流，minimum(s,t) 求最小净流；同一对象只能任选其一调用一次，s≠t。minimum 交换源汇求最大值后取反，原边方向及 used 不变；返回 optional<long long>，nullopt 表示无可行流。目标为源点流出减流入的净流量，两种查询都允许负答案，不能把 -1 直接当无解；minimum 不额外强制净流量非负。只在成功后读取 used(id) 的原边方案。先加 t→s 与 s→t 两条辅助边兼容负净流，再求可行循环流；保存净值后清空全部辅助残量边，最后在原图从 s 到 t 增广。最优值与增加量均受原边上界总和控制。允许自环、重边；构建和清理 O(n+m)，主体为两次 Dinic，额外空间 O(n+m)。依赖 BoundedCirculation 与 Dinic，二者内部字段只读，不应在求解后再加边或改流。

```
struct BoundedMaxFlow
{
    using ll = long long;
    BoundedCirculation b;
    ll total = 0;
    int m = 0;

    BoundedMaxFlow(int n) : b(n) {}

    int add(int u, int v, ll lo, ll hi)
    {
        assert(0 <= lo && lo <= hi && hi <= LLONG_MAX - total);
        total += hi;
        b.add(u, v, lo, hi);
        return m++;
    }

    optional<ll> solve(int s, int t)
    {
        assert(!b.solved && 1 <= s && s <= b.n && 1 <= t && t <= b.n && s != t);
        int first = b.g.e.size();
        int back = b.add(t, s, 0, total);
        int forward = b.add(s, t, 0, total);
        if (!b.solve()) return nullopt;
        __int128 ans = (__int128)b.used(back) - b.used(forward);
        for (int id = first; id < (int)b.g.e.size(); id += 2) b.g.change_edge(id, 0, 0);
        ans += b.g.flow(s, t);
        assert(-(__int128)total <= ans && ans <= total);
        return (ll)ans;
    }
```

最小净流与原边流量（接上页同一结构体）：

```
    optional<ll> minimum(int s, int t)
    {
        auto ans = solve(t, s);
        if (ans) *ans = -*ans;
        return ans;
    }

    ll used(int id) const
    {
        assert(0 <= id && id < m);
        return b.used(id);
    }
};
```

11.5 Stoer–Wagner 算法

11.5.1 无向全局最小割

solve(w)，n≥2；w 为 n×n 对称非负权矩阵，重边先累加，主对角置零。返回 {割权, 一侧顶点列表}，编号 0..n-1；两侧非空，不要求原图连通，不连通时割权为 0。并列最优割返回其中一种，不承诺点数或字典序。矩阵按值传入，原图不变。$O(n^3)$ 时间、$O(n^2)$ 空间；无向边总权（每条只计一次）严格小于 2^120，内部 int128。负权不适用。已逐条核验返回点集在原图上的跨割费用，暂无专门 OJ AC。

```
struct StoerWagner
{
    using I = __int128_t;

    // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
    static pair<I, vector<int>> solve(vector<vector<I>> w)
    {
        int n = w.size();
        assert(n >= 2);
        vector<int> active(n);
        iota(active.begin(), active.end(), 0);
        vector<vector<int>> group(n);
        for (int i = 0; i < n; i++) group[i] = {i};
        I best = I(1) << 120;
        vector<int> side;
        while (active.size() > 1)
        {
            vector<I> dis(n);
            vector<bool> used(n);
            int prev = -1;
            for (int step = 0; step < (int)active.size(); step++)
            {
                int sel = -1;
                for (int v : active)
                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
```

本轮割、合并与继续扩张（接上页同一函数）：

```
                if (step + 1 == (int)active.size())
                {
                    if (dis[sel] < best)
                    {
                        best = dis[sel];
                        side = group[sel];
                    }
                    for (int v : active)
                        if (v != prev && v != sel)
                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
                    group[prev].insert(
                        group[prev].end(), group[sel].begin(), group[sel].end());
                    active.erase(find(active.begin(), active.end(), sel));
                    break;
                }
                used[sel] = true;
                prev = sel;
                for (int v : active)
                    if (!used[v]) dis[v] += w[sel][v];
            }
        }
        return {best, side};
    }
};
```

模板题使用：Luogu P5632

题意：无向连通正权图的全局最小割，n 至 600、边权总和至 10^9；矩阵按无向边双向累加、自环忽略。核心返回费用与 0-based 割侧 side，本题只输出费用。核心要求 n≥2；单点没有非平凡割，驱动额外约定输出 0。

所需模板：StoerWagner（第 142 页，11.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5632

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using I = StoerWagner::I;
    vector<vector<I>> w(n, vector<I>(n));
    while (m--)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        u--;
        v--;
        if (u == v) continue;
        w[u][v] += c;
        w[v][u] += c;
    }
    // A singleton has no nontrivial cut; use zero by convention.
    if (n == 1)
    {
        cout << 0 << '\n';
        return 0;
    }
    auto [ans, side] = StoerWagner::solve(w);
    // The statement bounds the sum of edge weights by 1e9.
    cout << (long long)ans << '\n';
}
```

第十二章　图的匹配

12.1　二分图最大匹配

相关应用：正则多重二分图的随机交替游走；概率只影响耗时，不影响返回方案正确性。

12.1.1　正则二分图完美匹配

regular_matching(g, seed) 返回左侧各点的右侧匹配点，均为 0..n-1；g 为左侧邻接表，两侧每点度数均须等于同一正整数 d。重边按重数重复存储，不可去重；空图返回空。输入不修改。仅求一个完美匹配，不适用于一般非正则二分图或非空零度图。

从均匀随机未匹配左点开始，按邻接项均匀抽边，沿匹配边继续；path 和 pos 维护无环交替游走，遇旧左点时删去环。命中未匹配右点后倒序翻转；只在匹配全部建立后返回，不存在概率性错误答案。未匹配左点数为 k 时，每次尝试最多 4(floor(n/k)+2) 步，超限清空路径重试，不改变匹配。包含当前匹配边的采样项只形成自环，可正常消除。

理想独立均匀随机模型下，游走期望 O(n log(n+1))；加上全量度数检查，总期望 O(nd+n log(n+1))，额外空间 O(n)。固定 PRNG 种子没有确定性最坏耗时保证。seed 显式传入，测试用固定种子，赛时用时钟。随机性仅影响耗时与选择哪份正确方案；QOJ265 已以 CCF_NOI 满分通过，排名未核验。

期望界　将匹配点对收缩、原边定向左到右，加入源到未匹配左点和未匹配右点到汇各 d 条边，再识别源汇。所有点入出度平衡，源出度 kd，总出度 $(n+2k)d$；随机游走返回源的期望步数不超过 $n/k+2$。截断后重试的单次成功概率至少 $1/2$，故增广期望 $O(1+n/k)$，求和得 $O(n\log(n+1))$。路径数组只保存游走并消环，不是模拟 DFS 调用栈。

度数检查、未匹配集合与随机起步：

```
vector<int> regular_matching(const vector<vector<int>> &g, uint64_t seed)
{
    int n = g.size();
    if (!n) return {};
    int d = g[0].size();
    assert(d > 0);
    vector<int> degree(n);
    for (const auto &row : g)
    {
        assert((int)row.size() == d);
        for (int v : row)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
    }
    for (int value : degree) assert(value == d);
    vector<int> left(n, -1), right(n, -1), free(n), pos(n, -1), path;
    iota(free.begin(), free.end(), 0);
    mt19937_64 rng(seed);
    uniform_int_distribution<int> edge(0, d - 1);
    while (!free.empty())
    {
        int index = uniform_int_distribution<int>(0, free.size() - 1)(rng);
        int u = free[index];
        path.push_back(u);
        pos[u] = 0;
        long long limit = 4LL * (n / free.size() + 2);
        while (limit--)
        {
            int v = g[u][edge(rng)];
```

交替游走消环、增广与重试（接上页函数）：

```
            int next = right[v];
            if (next == -1)
            {
                for (int i = (int)path.size() - 1; i >= 0; i--)
                {
                    int x = path[i];
                    int old = left[x];
                    left[x] = v;
                    right[v] = x;
                    v = old;
                }
                free[index] = free.back();
                free.pop_back();
                break;
            }
            if (pos[next] == -1)
            {
                pos[next] = path.size();
                path.push_back(next);
            }
            else
            {
                while ((int)path.size() > pos[next] + 1)
                {
                    pos[path.back()] = -1;
                    path.pop_back();
                }
            }
            u = next;
        }
        for (int x : path) pos[x] = -1;
        path.clear();
    }
    return left;
}
```

模板题使用：QOJ265 正则二分图匹配

题意：两侧各 n 点、每点度数 d，允许重边；输出任一完美匹配，输入输出 1-based，函数内部 0-based。

所需模板：regular_matching（第 145 页，12.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/265

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, d;
    cin >> n >> d;
    vector<vector<int>> g(n, vector<int>(d));
    for (auto &row : g)
        for (int &v : row)
        {
            cin >> v;
            v--;
        }
    uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
    auto match = regular_matching(g, seed);
    for (int v : match) cout << v + 1 << ' ';
    cout << '\n';
}
```

12.1.2 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 1..n、1..m，可有空侧及重边；add(u,v) 加边，solve() 返回最大匹配总数，l[u] 与 r[v] 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve；不支持删边或手动更改匹配数组。BFS 用 dep[0] 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。V=n+m，单次 solve 最坏 $O((V+E)\sqrt{V})$，空间 O(V+E)。cover() 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
struct BipartiteMatching
{
    int n, m;
    vector<vector<int>> g;
    vector<int> l, r, dep, cur;

    BipartiteMatching(int n, int m)
        : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
    {
    }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        g[u].push_back(v);
    }

    bool bfs()
    {
        queue<int> q;
        fill(dep.begin(), dep.end(), n + 1);
```

```
        fill(cur.begin(), cur.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            if (!l[u])
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (dep[u] >= dep[0]) continue;
            for (int v : g[u])
            {
                int next = r[v];
                if (dep[next] != n + 1) continue;
                dep[next] = dep[u] + 1;
                if (next) q.push(next);
            }
        }
        return dep[0] != n + 1;
    }
```

增广、求解与最小点覆盖（接上页结构体）：

```
    bool dfs(int u)
    {
        if (!u) return true;
        for (int &i = cur[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
            {
                l[u] = v;
                r[v] = u;
                return true;
            }
        }
        dep[u] = n + 1;
        return false;
    }

    // Retains an existing matching; edges may be added between calls.
    int solve()
    {
        int answer = 0;
        for (int u = 1; u <= n; u++) answer += (l[u] != 0);
        while (bfs())
        {
            for (int u = 1; u <= n; u++)
            {
                if (!l[u] && dfs(u)) answer++;
            }
        }
        return answer;
    }

    // Call after solve(), before adding more edges. Returns left/right vertex IDs.
    pair<vector<int>, vector<int>> cover() const
    {
        vector<int> left, right;
        for (int u = 1; u <= n; u++)
        {
            if (dep[u] == n + 1) left.push_back(u);
        }
        for (int v = 1; v <= m; v++)
        {
            if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
        }
        return {left, right};
    }
};
```

模板题使用：Luogu P3386

题意：给定二分图，求最大匹配数。左右两侧分别编号 1..n、1..m，add(u,v) 的 u 属于左侧，v 属于右侧；solve 返回匹配数。此题不输出匹配或最小点覆盖方案。

所需模板：BipartiteMatching（第 148 页，12.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3386

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, e;
    std::cin >> n >> m >> e;
    BipartiteMatching graph(n, m);
    while (e--)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    std::cout << graph.solve() << '\n';
}
```

12.1.3 二分图最大匹配的可行边与必选边

代码依赖：BipartiteMatching（第 148 页，12.1.2 节）、TarjanSCC（第 84 页，6.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

matching_edges(n,m,edges) 接收左右两侧点号各为 1..n、1..m 的边表，返回 (最大匹配大小, 逐边分类)。每项为 {possible,forced}：是否属于至少一个/每一个最大匹配；分类保持输入顺序，平行边作为不同边处理。允许空侧、空图和非完备匹配。先用 BipartiteMatching 求解，再建含源汇的残量图跑 TarjanSCC；已匹配或两端同 SCC 为可行，已匹配且两端不同 SCC 为必选。不能在非完备匹配时省略源汇，也不能把可行条件写成已匹配且同 SCC。时间 $O(E\sqrt{}(n+m)+n+m)$，空间 O(n+m+E)，节点与残量边数须可用 int 表示。HK 与 SCC 都保留递归，栈深可为 O(n+m)。

```
pair<int, vector<array<bool, 2>>>
matching_edges(int n, int m, const vector<pair<int, int>> &edges)
{
    BipartiteMatching b(n, m);
    for (auto [u, v] : edges)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= m);
        b.add(u, v);
    }
    int size = b.solve(), s = n + m + 1, t = s + 1;
    TarjanSCC d(t);
    for (int u = 1; u <= n; u++)
        if (b.l[u])
            d.add(u, s);
        else
            d.add(s, u);
    for (int v = 1; v <= m; v++)
        if (b.r[v])
            d.add(t, n + v);
        else
            d.add(n + v, t);
    vector<bool> chosen(n + 1), used(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        used[i] = b.l[u] == v && !chosen[u];
        if (used[i])
        {
            chosen[u] = true;
            d.add(n + v, u);
        }
        else
            d.add(u, n + v);
    }
    d.run();
    vector<array<bool, 2>> ans(edges.size());
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        bool same = d.bel[u] == d.bel[n + v];
        ans[i] = {used[i] || same, used[i] && !same};
    }
    return {size, ans};
}
```

12.1.4 二分图最大独立集方案

代码依赖：BipartiteMatching（第 148 页，12.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

independent_set(g) 接收已建好的 BipartiteMatching，内部调用 solve 再取最小点覆盖的补集，返回左右两侧的原点号列表，分别从 1 计、升序排列。允许空侧、孤立点、重边与断开图；返回一种最大基数独立集，不保证字典序或最大权。只适用于已知左右划分的二分图，不能直接输入一般图或同侧边。

调用会更新 g 的匹配与搜索状态，保留合法旧匹配并继续增广；调用后可立即读 g.cover()。允许继续 add 后再次调用，不可删除边或手改匹配。n,m 非负且小于 INT_MAX，内存足够。最大独立集大小等于 n+m 减最大匹配数；只求数量可直接使用该式，求和时先转 long long。

依赖 BipartiteMatching；总时间 $O((V+E)\sqrt{(V+1)})$，V=n+m，额外时间与空间 O(V)。HK 增广保持递归，深度最坏 O(V)。骑士攻击使格子黑白染色反转，故可应用此函数；障碍格不要建成孤立点。

```
// Maximum independent set; left/right IDs are separate and 1-based.
pair<vector<int>, vector<int>> independent_set(BipartiteMatching &g)
{
    g.solve();
    auto [a, b] = g.cover();
    vector<bool> x(g.n + 1, true), y(g.m + 1, true);
    for (int u : a) x[u] = false;
    for (int v : b) y[v] = false;
    vector<int> left, right;
    for (int u = 1; u <= g.n; u++)
    {
        if (x[u]) left.push_back(u);
    }
    for (int v = 1; v <= g.m; v++)
    {
        if (y[v]) right.push_back(v);
    }
    return {left, right};
}
```

12.2 二分图最大权匹配

12.2.1 矩形二分图最小权匹配

solve(cost) 输入 n×m 完整权值矩阵，0≤n≤m；每行匹配不同列，返回最小费用及列号（0-based）。允许负费用，使用 int128 势能；$O(n^2m)$。禁止边不能用未经证明的有限大常数随意替代，当前接口仅表示完整二分图。缺边建模：求最小费用完美匹配时，禁止边代价取 C>n · (最大代价 − 最小代价)（C 须在 long long 内），任何不含禁止边的方案都比含禁止边的便宜；若最优解仍用到禁止边，则不存在完美匹配。求非完美的最大权匹配时，把权取负、缺边与负权边记 0，并补零列使 n≤m，选中零边即未匹配。

```
struct Hungarian
{
    using I = __int128_t;

    // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
    static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
    {
        int n = cost.size();
        if (!n) return {0, {}};
        int m = cost[0].size();
        assert(n <= m);
        vector<I> u(n + 1), v(m + 1);
```

```
        vector<int> p(m + 1), way(m + 1);
        const I inf = I(1) << 120;
        for (int i = 1; i <= n; i++)
        {
            assert((int)cost[i - 1].size() == m);
            p[0] = i;
            int j0 = 0;
            vector<I> dis(m + 1, inf);
            vector<bool> used(m + 1);
            do
            {
                used[j0] = true;
                int i0 = p[j0], j1 = 0;
                I delta = inf;
                for (int j = 1; j <= m; j++)
                    if (!used[j])
                    {
                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
                        if (cur < dis[j])
                        {
                            dis[j] = cur;
                            way[j] = j0;
                        }
                        if (dis[j] < delta)
                        {
                            delta = dis[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j <= m; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta;
                        v[j] -= delta;
                    }
                    else
                        dis[j] -= delta;
                j0 = j1;
            } while (p[j0]);
            do
            {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0);
        }
        vector<int> ans(n);
        for (int j = 1; j <= m; j++)
            if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans};
    }
};
```

模板题使用：Library Checker assignment

题意：给完整方阵，为每一行分配一个不同的列，使费用和最小。返回列号是 0-based，按行顺序输出。题目 n≤500、单项绝对值 ≤10^9，总费用可由 int128 转为 long long；缺边问题不能直接套本例。

所需模板：Hungarian（第 153 页，12.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/assignment

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<long long>> cost(n, vector<long long>(n));
    for (auto &row : cost)
        for (auto &x : row) cin >> x;
    auto [value, columns] = Hungarian::solve(cost);
    cout << (long long)value << '\n';
    for (int j : columns) cout << j << ' ';
    cout << '\n';
}
```

12.3 一般图最大匹配

12.3.1 一般图最大基数匹配：带花树

Blossom(n)，点号 0..n-1；add(u,v) 加无向边，忽略自环，允许重边。solve() 从空匹配重新计算，返回匹配边数；mate[u] 为配对点，未匹配为 -1。结果支持直接输出方案。缩花维护交错树，单次增广 $O(n^2+m)$，总计 $O(n^3+nm)$，简单图为 $O(n^3)$；空间 O(n+m)。重复调用 solve 或加边后重算均可。不处理边权，不能代替最大权匹配。

```
struct Blossom
{
    int n;
    vector<vector<int>> g;
    vector<int> mate, pre, base, q;
    vector<bool> outer, flower, seen;

    Blossom(int n)
        : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
    {
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), false);
        while (true)
```

```
        {
            u = base[u];
            seen[u] = true;
            if (mate[u] == -1) break;
            u = pre[mate[u]];
        }
        while (!seen[base[v]]) v = pre[mate[base[v]]];
        return base[v];
    }

    void mark(int u, int b, int v)
    {
        while (base[u] != b)
        {
            flower[base[u]] = true;
            flower[base[mate[u]]] = true;
            pre[u] = v;
            v = mate[u];
            u = pre[v];
        }
    }

    bool augment(int root)
    {
        fill(outer.begin(), outer.end(), false);
        fill(pre.begin(), pre.end(), -1);
        iota(base.begin(), base.end(), 0);
        q.assign(1, root);
        outer[root] = true;
        for (int head = 0; head < (int)q.size(); head++)
        {
            int u = q[head];
            for (int v : g[u])
            {
                if (base[u] == base[v] || mate[u] == v) continue;
                if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
                {
                    int b = lca(u, v);
                    fill(flower.begin(), flower.end(), false);
                    mark(u, b, v);
                    mark(v, b, u);
                    for (int x = 0; x < n; x++)
                    {
                        if (!flower[base[x]]) continue;
                        base[x] = b;
                        if (!outer[x])
                        {
                            outer[x] = true;
                            q.push_back(x);
                        }
                    }
                }
                else if (pre[v] == -1)
                {
                    pre[v] = u;
                    if (mate[v] == -1)
```

```
                    {
                        while (v != -1)
                        {
                            int p = pre[v];
                            int next = mate[p];
                            mate[v] = p;
                            mate[p] = v;
                            v = next;
                        }
                        return true;
                    }
                    int x = mate[v];
                    outer[x] = true;
                    q.push_back(x);
                }
            }
        }
        return false;
    }

    // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
    int solve()
    {
        fill(mate.begin(), mate.end(), -1);
        int answer = 0;
        for (int u = 0; u < n; u++)
        {
            if (mate[u] == -1 && augment(u)) answer++;
        }
        return answer;
    }
};
```

模板题使用：Library Checker general_matching

题意：给简单无向图，输出最大匹配的边数及匹配边。点号 0..n-1；solve 后 mate[u]=-1 表示未匹配，只在 u<mate[u] 时输出，避免同一条边打印两次。

所需模板：Blossom（第 155 页，12.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/general_matching

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Blossom graph(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    cout << graph.solve() << '\n';
    for (int u = 0; u < n; u++)
        if (u < graph.mate[u]) cout << u << ' ' << graph.mate[u] << '\n';
}
```

12.4 一般图最大权匹配

12.4.1 一般图最大权匹配：带权带花树

WeightedBlossom(n)，点 1..n，0≤n<INT_MAX/2。add_edge(u,v,w) 添加 signed64 边权，自环忽略，重边取最大。solve() 只最大化总权，允许空匹配，不保证同权时边数最多；solve(true) 先最大化匹配边数，再最大化原权。返回 weight（__int128）、pairs、mate（未匹配为 0）。求最大权完美匹配用 solve(true)，并检查 2*pairs==n。

每次 solve 从头计算，支持重复、切换模式、加边后重算。$O(n^3)$ 时间、$O(n^2)$ 空间，稠密矩阵适合数百点；不因接口允许大 n 而保证能装入内存。内部成员仅供核验，不应直接修改。缩花、展开与增广使用递归；全范围 signed64 权值以 128 位对偶计算。

改编自 Shahjalal Shohag 的 MIT 实现，完整许可见后页。P6699 已 100 分通过；负权与基数优先模式由独立本地验证覆盖。

字段与加边（同一结构体内连续抄写）：

```
struct WeightedBlossom
{
    using I = __int128;
    using ll = long long;
    static constexpr I INF = I(1) << 126;
    struct Edge
    {
        int u = 0, v = 0;
        I w = 0;
    };
    struct Result
    {
        I weight = 0;
        int pairs = 0;
        vector<int> mate;
    };
    int n, top = 0;
    vector<vector<I>> weight;
    vector<vector<Edge>> g;
    vector<vector<int>> flower, from;
    vector<int> match, slack, base, parent, state, seen;
    vector<I> label;
    queue<int> q;

    explicit WeightedBlossom(int n) : n(n), weight(n + 1, vector<I>(n + 1, -INF))
    {
        assert(0 <= n && n < INT_MAX / 2);
    }
    void add_edge(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return;
        weight[u][v] = max(weight[u][v], I(w));
        weight[v][u] = weight[u][v];
    }
```

松弛与递归代表元（同一结构体内连续抄写）：

```
    I gap(const Edge &e) const
    {
        return label[e.u] + label[e.v] - 2 * e.w;
    }
    void relax(int u, int x)
    {
        if (!slack[x] || gap(g[u][x]) < gap(g[slack[x]][x])) slack[x] = u;
    }
    void reset_slack(int x)
    {
        slack[x] = 0;
        for (int u = 1; u <= n; u++)
            if (g[u][x].w > 0 && base[u] != x && state[base[u]] == 0)
                relax(u, x);
    }
    void enqueue(int x)
    {
        if (x <= n) q.push(x);
        else for (int v : flower[x]) enqueue(v);
    }
    void set_base(int x, int b)
    {
        base[x] = b;
        if (x > n)
            for (int v : flower[x]) set_base(v, b);
    }
    int position(int b, int x)
    {
        int k = find(flower[b].begin(), flower[b].end(), x) - flower[b].begin();
        if (k % 2 == 0) return k;
        reverse(flower[b].begin() + 1, flower[b].end());
        return flower[b].size() - k;
    }
```

花内配对、增广与公共祖先（同一结构体内连续抄写）：

```
    void pair_with(int u, int v)
    {
        Edge e = g[u][v];
        match[u] = e.v;
        if (u <= n) return;
        int x = from[u][e.u];
        int k = position(u, x);
        for (int i = 0; i < k; i++) pair_with(flower[u][i], flower[u][i ^ 1]);
        pair_with(x, v);
        rotate(flower[u].begin(), flower[u].begin() + k, flower[u].end());
    }
    void augment(int u, int v)
    {
        int next = base[match[u]];
        pair_with(u, v);
        if (!next) return;
        pair_with(next, base[parent[next]]);
        augment(base[parent[next]], next);
    }
    int lca(int u, int v)
    {
        fill(seen.begin(), seen.end(), 0);
        while (u || v)
        {
            if (u)
            {
                if (seen[u]) return u;
                seen[u] = 1;
                u = base[match[u]];
                if (u) u = base[parent[u]];
            }
            swap(u, v);
        }
        return 0;
    }
```

缩花及原始端点映射（同一结构体内连续抄写）：

```
104 void shrink(int u, int root, int v)
105 {
106     int b = n + 1;
107     while (b <= top && base[b]) b++;
108     if (b > top) top++;
109     label[b] = 0;
110     state[b] = 0;
111     match[b] = match[root];
112     flower[b] = {root};
113     for (int end : {u, v})
114     {
115         for (int x = end; x != root; )
116         {
117             int y = base[match[x]];
118             flower[b].push_back(x);
119             flower[b].push_back(y);
120             enqueue(y);
121             x = base[parent[y]];
122         }
123         if (end == u) reverse(flower[b].begin() + 1, flower[b].end());
124     }
125     set_base(b, b);
126     for (int x = 1; x <= top; x++)
127     {
128         g[b][x] = {};
129         g[x][b] = {};
130     }
131     fill(from[b].begin(), from[b].end(), 0);
132     for (int child : flower[b])
133     {
134         for (int x = 1; x <= top; x++)
135             if (g[child][x].w > 0 &&
136                 (!g[b][x].w || gap(g[child][x]) < gap(g[b][x])))
137             {
138                 g[b][x] = g[child][x];
139                 g[x][b] = g[x][child];
140             }
141         for (int x = 1; x <= n; x++)
142             if (from[child][x]) from[b][x] = child;
143     }
144     reset_slack(b);
145 }
```

展开与紧边处理（同一结构体内连续抄写）：

```
    void expand(int b)
    {
        for (int x : flower[b]) set_base(x, x);
        int entry = from[b][g[b][parent[b]].u];
        int k = position(b, entry);
        for (int i = 0; i < k; i += 2)
        {
            int x = flower[b][i];
            int y = flower[b][i + 1];
            parent[x] = g[y][x].u;
            state[x] = 1;
            state[y] = 0;
            slack[x] = 0;
            reset_slack(y);
            enqueue(y);
        }
        state[entry] = 1;
        parent[entry] = parent[b];
        for (int i = k + 1; i < (int)flower[b].size(); i++)
        {
            int x = flower[b][i];
            state[x] = -1;
            reset_slack(x);
        }
        base[b] = 0;
    }
    bool use_edge(Edge e)
    {
        int u = base[e.u];
        int v = base[e.v];
        if (state[v] == -1)
        {
            parent[v] = e.u;
            state[v] = 1;
            int next = base[match[v]];
            slack[v] = 0;
            slack[next] = 0;
            state[next] = 0;
            enqueue(next);
        }
        else if (state[v] == 0)
        {
            int root = lca(u, v);
            if (root) shrink(u, root, v);
            else
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
        }
        return false;
    }
```

交替森林搜索（同一结构体内连续抄写）：

```
bool search()
{
    fill(state.begin(), state.end(), -1);
    fill(slack.begin(), slack.end(), 0);
    q = {};
    for (int x = 1; x <= top; x++)
        if (base[x] == x && !match[x])
        {
            parent[x] = 0;
            state[x] = 0;
            enqueue(x);
        }
    if (q.empty()) return false;
    while (true)
    {
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (state[base[u]] == 1) continue;
            for (int v = 1; v <= n; v++)
                if (g[u][v].w > 0 && base[u] != base[v])
                {
                    if (gap(g[u][v]) == 0)
                    {
                        if (use_edge(g[u][v])) return true;
                    }
                    else relax(u, base[v]);
                }
        }
```

对偶更新与继续搜索（同一结构体内连续抄写）：

```
            I delta = INF;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1)
                    delta = min(delta, label[b] / 2);
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x])
                {
                    I d = gap(g[slack[x]][x]);
                    if (state[x] == -1) delta = min(delta, d);
                    if (state[x] == 0) delta = min(delta, d / 2);
                }
            for (int u = 1; u <= n; u++)
                if (state[base[u]] == 0) delta = min(delta, label[u]);
            bool finished = false;
            for (int u = 1; u <= n; u++)
            {
                if (state[base[u]] == 0)
                {
                    label[u] -= delta;
                    if (label[u] == 0) finished = true;
                }
                else if (state[base[u]] == 1) label[u] += delta;
            }
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b)
                {
                    if (state[b] == 0) label[b] += 2 * delta;
                    if (state[b] == 1) label[b] -= 2 * delta;
                }
            if (finished) return false;
            q = {};
            for (int x = 1; x <= top; x++)
                if (base[x] == x && slack[x] && base[slack[x]] != x &&
                    gap(g[slack[x]][x]) == 0)
                    if (use_edge(g[slack[x]][x])) return true;
            for (int b = n + 1; b <= top; b++)
                if (base[b] == b && state[b] == 1 && label[b] == 0) expand(b);
        }
    }
```

两种目标、初始化与方案返回（同一结构体内连续抄写）：

```
    Result solve(bool cardinality_first = false)
    {
        int size = 2 * n + 1;
        top = n;
        g.assign(size, vector<Edge>(size));
        flower.assign(size, {});
        from.assign(size, vector<int>(n + 1));
        match.assign(size, 0);
        slack.assign(size, 0);
        base.assign(size, 0);
        parent.assign(size, 0);
        state.assign(size, -1);
        seen.assign(size, 0);
        label.assign(size, 0);
        I bound = 0;
        for (int u = 1; u <= n; u++)
            for (int v = u + 1; v <= n; v++)
                if (weight[u][v] != -INF) bound = max(bound, max(weight[u][v], -weight[u][v]));
        I bonus = cardinality_first ? n * bound + 1 : 0;
        I maximum = 0;
        for (int u = 1; u <= n; u++)
        {
            base[u] = u;
            from[u][u] = u;
            for (int v = 1; v <= n; v++)
                if (weight[u][v] != -INF && weight[u][v] + bonus > 0)
                {
                    g[u][v] = {u, v, weight[u][v] + bonus};
                    maximum = max(maximum, g[u][v].w);
                }
        }
        fill(label.begin() + 1, label.begin() + n + 1, maximum);
        Result result;
        for (int u = 1; u <= n; u++)
            if (!match[u] && maximum > 0)
                for (int v = u + 1; v <= n; v++)
                    if (!match[v] && g[u][v].w == maximum)
                    {
                        match[u] = v;
                        match[v] = u;
                        result.pairs++;
                        break;
                    }
        while (search()) result.pairs++;
        result.mate.assign(match.begin(), match.begin() + n + 1);
        for (int u = 1; u <= n; u++)
            if (result.mate[u] > u) result.weight += weight[u][result.mate[u]];
        return result;
    }
};
```

目标与证书 基数优先模式给每条存在边加 $B = nW + 1$，W 为最大绝对原权；任意两个匹配原权差不超过 nW，所以多一条边严格占优。求完美匹配必须选择此模式再检查 $2 \cdot pairs = n$。顶点标签与奇集标签给出匹配 LP 的对偶上界；边约束可行、匹配边紧、未匹配顶点标签零以及正标签花饱和使上下界相等。所有标签应完成更新后再终止，不能留下部分更新。

验证范围 六种构建各 28546 次子集 DP、方案和对偶检查；400 点 16 次大图证书检查通过 O2 与 ASan/UBSan。实际触发嵌套缩花与展开。P6699 记录 302632540，20 点全 AC，最慢 209ms、最高 24.02MB；负权、基数优先及完美判断为本地证据，排名未核验。

来源及 MIT 许可 改编自 Shahjalal Shohag 的带权带花树，固定提交 0884be6ab81f（完整提交号见仓库来源记录）。以下许可随本实现保留：

Copyright (c) 2021 Shahjalal Shohag License: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the ”Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

模板题使用：P6699 一般图最大权匹配

题意：n≤400、m≤79800、1≤w≤5×10^8。输出最大总权及每点的匹配点，未匹配为 0。答案 ≤10^{11}，可转 long long；方案不唯一。当前源码记录 302632540，20 点全 AC。

所需模板：WeightedBlossom（第 159 页，12.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6699

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    WeightedBlossom g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add_edge(u, v, w);
    }
    auto result = g.solve();
    cout << (long long)result.weight << '\n';
    for (int u = 1; u <= n; u++) cout << result.mate[u] << ' ';
    cout << '\n';
}
```

第十三章　Prüfer 序列

13.1　Prüfer 双射与有标号树计数

顶点标号 $1,\dots,n$ 的有标号树，$n\ge 2$：反复删最小标号叶子并记录其邻点，直到剩两点，得长度 $n-2$ 的序列。$n=2$ 为空序列；$n=1$ 单独处理，不代入含 $n-2$ 的公式。

13.1.1　双射为何成立

解码：读下一项 u，取未删除且不在剩余序列中的最小标号 v，连 uv 并删 v；最后连剩下两点。

$$d_i = 1 + \#\{\text{序列中等于} i \text{ 的项}\}.$$

13.1.2　Cayley、指定度数与叶子数

树总数 n^{n-2}。度数 $d_1,\dots,d_n$：若某 $d_i<1$ 或 $\sum_i d_i \ne 2n-2$ 则为 0，否则

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

恰有 ℓ 个叶子：

$$\binom{n}{\ell}\,(n-\ell)!\,S(n-2,n-\ell),$$

$n\ge 3$ 时 $2\le \ell\le n-1$，其余为 0；$n=2,\ell=2$ 得 1。取模时阶乘分母须可逆，素数模见第 B.1.1 节（第 191 页）。

13.1.3 加权 Cayley 与连接已有连通块

完全图边权 $w_{ij} = x_i x_j$ 时

$$\sum_T \prod_{ij \in T} w_{ij} = \sum_T \prod_i x_i^{d(i)} = \left(\prod_i x_i\right)\left(\sum_i x_i\right)^{n-2},$$

对零权、负权及任意正模数成立；$n = 2$ 时零次幂为 1。一般边权或缺边图用矩阵树定理，见第 14.3 节（第 178 页）及第 14.2 节（第 176 页）。

$k \geq 2$ 个连通块大小 $s_1, \ldots, s_k$，$N = \sum_i s_i$，恰加 $k-1$ 条块间边使其连通的无序边集数为

$$N^{k-2} \prod_{i=1}^{k} s_i;$$

$k = 1$ 时为 1。禁止部分端点对、限制新增边或计执行顺序时不适用。有标号树数不能除以 $n!$ 得无标号树数。

第十四章　矩阵树定理

相关应用：生成树计数的相关扩展：Tutte 多项式，T(1,1) 对应连通图生成树数；不将其冒称为矩阵树定理实现。

14.1　Tutte 多项式求值

tutte(adj,x,y) 返回模 998244353 的 Tutte 多项式值。简单无向图，n≤21，点号 0..n-1；adj[u] 为邻居位掩码，须对称、无自环、无越界位。0≤x,y<998244353；支持断连、孤立点及空图（值 1），不支持重边或自环；输入不修改。连通分量 DFS 保持递归。

每个连通分量独立计算并相乘。c[S] 为 S 上连通生成子图按 (y-1) 的环秩计权之和；删去 S 的最小顶点 r 后，每个剩余连通块 T 的连接权为 1+y+...+y^(h-1)，h 为 r 连向 T 的边数，h=0 时为 0。用快速子集指数求集合划分。最后固定顶点 0 所在块，其余各块乘 (x-1)。不除 x-1 或 y-1，x=1、y=1 均直接支持；普通形式指数只需 1..20 的模逆元。时间 O(n^{2}2^n)，峰值空间 O(n2^n)。21 点 Linux 九次连续求值进程峰值约 312MiB；QOJ45 已以 CCF_NOI 获得满分；排名尚未核验。

集合指数为何正确　按集合大小分次数层，zeta 后做普通形式指数，再 Mobius 并取集合大小次项。逆变换只保留并集恰为 S 的项，次数等于 $|S|$ 排除重叠；$1/k!$ 抵消 k 个块的排列。最小顶点从大到小处理时，余下块的 c 值已知。最终固定 0 所在块，其他块各乘 $x-1$，恰好给出定义中的连通分量指数。

输入约定、等比和与分层 zeta：

```
int tutte(const vector<unsigned> &adj, int x, int y)
{
    constexpr int mod = 998244353;
    int n = adj.size();
    assert(n <= 21 && 0 <= x && x < mod && 0 <= y && y < mod);
    for (int u = 0; u < n; u++)
    {
        assert((adj[u] >> n) == 0 && !(adj[u] >> u & 1));
        for (int v = 0; v < n; v++)
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1));
    }
    vector<int> inv(n + 1, 1), geometric(n + 1);
    for (int i = 2; i <= n; i++)
        inv[i] = mod - (long long)(mod / i) * inv[mod % i] % mod;
    for (int i = 1; i <= n; i++)
        geometric[i] = ((long long)geometric[i - 1] * y + 1) % mod;
    auto exponential = [&](const vector<int> &a)
    {
        int size = a.size();
        int k = countr_zero((unsigned)size), width = k + 1;
        vector<int> f((size_t)size * width), g(width), answer(size);
        for (int s = 1; s < size; s++) f[(size_t)s * width + popcount((unsigned)s)] = a[s];
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 1; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value += f[(size_t)(s ^ bit) * width + j];
                        if (value >= mod) value -= mod;
                    }
```

形式指数与 Mobius（接上页函数）：

```
        for (int s = 0; s < size; s++)
        {
            int *row = f.data() + (size_t)s * width;
            g[0] = 1;
            for (int i = 1; i <= k; i++) row[i] = (long long)row[i] * i % mod;
            for (int i = 1; i <= k; i++)
            {
                long long sum = 0;
                for (int j = 1; j <= i; j++)
                    sum = (sum + (long long)row[j] * g[i - j]) % mod;
                g[i] = sum * inv[i] % mod;
            }
            copy(g.begin(), g.end(), row);
        }
        for (int bit = 1; bit < size; bit <<= 1)
            for (int s = 0; s < size; s++)
                if (s & bit)
                    for (int j = 0; j <= k; j++)
                    {
                        int &value = f[(size_t)s * width + j];
                        value -= f[(size_t)(s ^ bit) * width + j];
                        if (value < 0) value += mod;
                    }
        for (int s = 0; s < size; s++) answer[s] = f[(size_t)s * width + popcount((unsigned)s)];
        return answer;
    };
```

连通子集递推及分量乘积（接上页函数）：

```
    auto connected = [&](const vector<unsigned> &g)
    {
        int m = g.size();
        int size = 1 << m;
        vector<int> c(size);
        for (int root = m - 1; root >= 0; root--)
        {
            int count = 1 << (m - root - 1);
            vector<int> a(count);
            for (int s = 1; s < count; s++)
            {
                unsigned t = (unsigned)s << (root + 1);
                a[s] = (long long)c[t] * geometric[popcount(g[root] & t)] % mod;
            }
            auto e = exponential(a);
            for (int s = 0; s < count; s++) c[(s << (root + 1)) | (1 << root)] = e[s];
        }
        if (x == 1) return c.back();
        vector<int> a(size / 2);
        int z = x ? x - 1 : mod - 1;
        for (int s = 1; s < size / 2; s++) a[s] = (long long)c[s << 1] * z % mod;
        auto e = exponential(a);
        long long answer = 0;
        for (int s = 0; s < size / 2; s++)
            answer = (answer + (long long)c[(size - 1) ^ (s << 1)] * e[s]) % mod;
        return (int)answer;
    };
    vector<int> seen(n), vertices;
    auto dfs = [&](auto &&self, int u) -> void
    {
        seen[u] = 1;
        vertices.push_back(u);
        for (int v = 0; v < n; v++)
            if ((adj[u] >> v & 1) && !seen[v]) self(self, v);
    };
    long long answer = 1;
    for (int u = 0; u < n; u++)
        if (!seen[u])
        {
            vertices.clear();
            dfs(dfs, u);
            vector<unsigned> g(vertices.size());
            for (int i = 0; i < (int)vertices.size(); i++)
                for (int j = 0; j < (int)vertices.size(); j++)
                    if (adj[vertices[i]] >> vertices[j] & 1) g[i] |= 1U << j;
            answer = answer * connected(g) % mod;
        }
    return answer;
}
```

模板题使用：QOJ45 Tutte 多项式

题意：输入简单无向图邻接矩阵和 x,y，输出模 998244353 的 Tutte 多项式值；n≤21。

所需模板：tutte（第 171 页，14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/45

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<unsigned> adj(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int edge;
            cin >> edge;
            if (edge) adj[u] |= 1U << v;
        }
    int x, y;
    cin >> x >> y;
    cout << tutte(adj, x, y) << '\n';
}
```

14.2 任意正模数带权矩阵树

代码依赖：determinant_mod（第 192 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(n, edges, root, kind, mod) 返回树的边权乘积之和对 mod 的最小非负余数。mod 为任意正 signed 64 位数，边权支持完整 signed 64 位。Kind 的 undirected、toward_root、away_from_root 分别表示无向、内向和外向树；点号 0..n-1。自环忽略，重边按不同边计数，单点返回 1%mod，mod=1 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 determinant_mod；$O(n^3 \log \mathrm{mod} + m)$ 时间、$O(n^2)$ 额外空间。不使用模逆元，素数与合数模数通用。

```
struct MatrixTreeMod
{
    using ll = long long;
    using Edge = tuple<int, int, ll>;
    enum class Kind
    {
        undirected,
        toward_root,
        away_from_root
    };

    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
    {
        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
        auto add = [&](int u, int v, ll w)
        {
            if (u == root) return;
            int i = u - (u > root), j = v - (v > root);
            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
        };
        for (auto [u, v, w] : edges)
        {
            assert(u >= 0 && u < n && v >= 0 && v < n);
            if (u == v) continue;
            w %= mod;
            if (w < 0) w += mod;
            if (kind == Kind::away_from_root) swap(u, v);
            add(u, v, w);
            if (kind == Kind::undirected) add(v, u, w);
        }
        return determinant_mod(move(lap), mod);
    }
};
```

模板题使用：Library Checker Counting Spanning Trees (Undirected)

题意：无向多重图生成树计数，模 998244353；点号 0..n-1，1≤n≤500、0≤m≤500000，允许自环和重边。每条原边权重设 1，自环不参与树，重边是不同选择。删去任意根的拉普拉斯行列，这里取根 0；n=1 的空树计 1，断连计 0。此接口也支持合数模数，复杂度 $O(n^3 \log \mathrm{mod})$，本题固定质数亦可选更快的 MatrixTree 版本。

所需模板：determinant_mod（第 192 页，C.1.1 节）、MatrixTreeMod（第 176 页，14.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/counting_spanning_tree_undirected

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<MatrixTreeMod::Edge> e;
    e.reserve(m);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        e.emplace_back(u, v, 1);
    }
    cout << MatrixTreeMod::count(n, e, 0, MatrixTreeMod::Kind::undirected, 998244353)
         << '\n';
}
```

14.3 带权矩阵树定理与方向约定

结果为所有生成树的边权乘积之和；无权计数令边权为 1。重边是不同选择，自环不属于任何生成树，单点为 1。零权、负权按代数运算处理，结果为零不能作为连通性判定。

无向边 uv 权 w：L_{uu}, L_{vv} 各加 w，L_{uv}, L_{vu} 各减 w；删除 root 行列求行列式，与 root 无关。内向树形图（每点沿边走到根）：边 $u \to v$ 贡献 L_{uu} += w、L_{uv} −= w，删根行列。外向树形图先反转所有边再用同一构造。只构造 $(n-1)\times(n-1)$ 余子式，时间 $O(n^3+m)$、额外空间 $O(n^2)$；恒等式不需要除法，模数须为素数只是模逆元消元的要求。

14.4 任意正模数的矩阵树计数

实现见第 14.2 节（第 176 页）。沿用第 14.3 节的三种构造，行列式改用欧几里得消元（第 C.1.1 节，第 192 页）。接口 `MatrixTreeMod::count`，边权和正模数均为 signed 64 位；时间 $O(n^3 \log(m+1) + |E|)$、空间 $O(n^2)$，m 为模数。边权先正规化到 $[0, m)$，矩阵加减先扩展到 128 位再取模；重边权和不能先累加到普通整数。单点返回 $1 \bmod m$，模 1 返回零；返回零不代表没有生成树。

第十五章　LGV 引理

15.1　LGV：DAG 路径权值行列式

代码依赖：ModInt（第 186 页，A.1.1 节）、determinant_mod（第 192 页，C.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

dag_path_determinant<mod>(g,sources,sinks) 返回路径权值矩阵的行列式，mod 必须为素数。g[u] 的项为 (v,ModInt<mod> 权值)，点号 0..n-1；两个端点序列等长 k，各自不能重复，但可相互重叠。允许重边、零权、负权；长度为 0 的路径权为 1。整张图有环则返回 nullopt，空端点族在 DAG 上返回 1。由 LGV，它是顶点不交路径族按汇点排列带符号的权值和；只有能证明非恒等排列均不可能时，才是对应配对的普通计数。顺序不能随意排序，模结果为 0 也不能据此断言不存在路径族。时间 $O((k+1)(n+m)+k^3+k \log mod)$，额外空间 $O(n+k^2)$，依赖 det_prime 和 ModInt。

```
template <int mod>
optional<ModInt<mod>>
dag_path_determinant(const vector<vector<pair<int, ModInt<mod>>>> &g,
                     const vector<int> &sources,
                     const vector<int> &sinks)
{
    using Z = ModInt<mod>;
    int n = g.size(), k = sources.size();
    assert((int)sinks.size() == k);
    vector<int> in(n), order;
    vector<bool> seen_source(n), seen_sink(n);
    for (int i = 0; i < k; i++)
    {
        assert(0 <= sources[i] && sources[i] < n);
        assert(0 <= sinks[i] && sinks[i] < n);
        assert(!seen_source[sources[i]] && !seen_sink[sinks[i]]);
        seen_source[sources[i]] = seen_sink[sinks[i]] = true;
    }
    for (const auto &edges : g)
        for (auto [v, w] : edges)
        {
            assert(0 <= v && v < n);
            in[v]++;
        }
    for (int u = 0; u < n; u++)
        if (!in[u]) order.push_back(u);
    for (int i = 0; i < (int)order.size(); i++)
        for (auto [v, w] : g[order[i]])
            if (!--in[v]) order.push_back(v);
    if ((int)order.size() != n) return nullopt;
```

```
    vector<vector<long long>> a(k, vector<long long>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[sources[i]] = 1;
        for (int u : order)
            for (auto [v, w] : g[u]) dp[v] = dp[v] + dp[u] * w;
        for (int j = 0; j < k; j++) a[i][j] = dp[sinks[j]].v;
    }
    return Z(determinant_mod(move(a), mod));
}
```

第十六章　支配树

16.1　支配树：Lengauer–Tarjan

DominatorTree(n)，点号 1..n；add(u,v) 加有向边，允许重边、自环。build(root) 后 idom 为原点号直接支配点，根指向自身，不可达为 0。仅对 root 可达点定义支配关系：所有 root 到 v 的路径均经过 u，等价于 u 在支配树上是 v 的祖先。timer 是可达点数，vertex[1..timer] 为 DFS 顺序，dfn 为逆映射；逆序向 idom 累加可得支配点数，根不向自身累加。g 保留原点号；pred、bucket、semi 等辅助数组使用 DFS 编号。可更换根重建；加边使预处理失效，需重新 build。简单 LT 时间 O((n+m) log(n+1))，空间及递归栈 O(n+m) 与 O(n)，n+1、边数须可由 int 表示。一般图与 DAG 均适用；可由 idom 建树组合 LCA。

```
struct DominatorTree
{
    int n, timer = 0;
    vector<vector<int>> g, pred, bucket;
    vector<int> dfn, vertex, parent, semi, label, ancestor, dom, idom;

    DominatorTree(int n) : n(n), g(n + 1) { assert(n > 0); }

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
        idom.clear();
    }

    void dfs(int u)
    {
        dfn[u] = ++timer;
        vertex[timer] = u;
        for (int v : g[u])
            if (!dfn[v])
            {
                dfs(v);
                parent[dfn[v]] = dfn[u];
            }
    }

    void compress(int v)
    {
        int a = ancestor[v];
        if (ancestor[a])
        {
            compress(a);
            if (semi[label[a]] < semi[label[v]]) label[v] = label[a];
            ancestor[v] = ancestor[a];
        }
    }
```

路径查询、半支配点与直接支配点计算（接上页同一结构体）：

```
    int eval(int v)
    {
        compress(v);
        return label[v];
    }

    void build(int root)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dfn.assign(n + 1, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        ancestor.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        idom.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        semi.resize(n + 1);
        iota(semi.begin(), semi.end(), 0);
        label = semi;
        dfs(root);
        for (int u = 1; u <= n; u++)
            if (dfn[u])
                for (int v : g[u]) pred[dfn[v]].push_back(dfn[u]);
        for (int i = timer; i >= 2; i--)
        {
            for (int p : pred[i]) semi[i] = min(semi[i], semi[eval(p)]);
            bucket[semi[i]].push_back(i);
            ancestor[i] = parent[i];
            for (int v : bucket[parent[i]])
            {
                int u = eval(v);
                dom[v] = semi[u] < semi[v] ? u : parent[i];
            }
            bucket[parent[i]].clear();
        }
        for (int i = 2; i <= timer; i++)
        {
            if (dom[i] != semi[i]) dom[i] = dom[dom[i]];
            idom[vertex[i]] = vertex[dom[i]];
        }
        idom[root] = root;
    }
};
```

模板题使用：Library Checker dominatortree

题意：从指定起点求每个点的直接支配点。输入输出是 0-based，调用模板时加 1；输出减 1 后，不可达点为 -1，根仍指向自身。

所需模板：DominatorTree（第 182 页，16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/dominatortree

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    DominatorTree t(n);
    while (m--)
    {
        int u, v;
        cin >> u >> v;
        t.add(u + 1, v + 1);
    }
    t.build(s + 1);
    for (int u = 1; u <= n; u++) cout << t.idom[u] - 1 << (u == n ? '\n' : ' ');
}
```

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
77      ModInt operator/(ModInt b) const
78      {
79          return *this * b.inv();
80      }
81
82      ModInt &operator+=(ModInt b)
83      {
84          return *this = *this + b;
85      }
86
87      ModInt &operator-=(ModInt b)
88      {
89          return *this = *this - b;
90      }
91
92      ModInt &operator*=(ModInt b)
93      {
94          return *this = *this * b;
95      }
96
97      ModInt &operator/=(ModInt b)
98      {
99          return *this = *this / b;
100     }
101 };
```

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 189 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

代码依赖：ModInt（第 186 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }

    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

附录 C　线性代数

C.1　行列式

C.1.1　任意正模数行列式：欧几里得消元

determinant_mod(a,mod) 返回 det(a) 的最小非负余数。a 为 long long 方阵，元素支持完整 signed 64 位；1≤mod≤LLONG_MAX，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 1%mod，mod=1 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 int128。O(n^3 log mod) 时间、O(n^2) 空间；素数模通常优先用已有高斯行列式。

```
long long determinant_mod(vector<vector<long long>> a, long long mod)
{
    assert(mod >= 1);
    int n = a.size();
    for (auto &row : a)
    {
        assert((int)row.size() == n);
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0) x += mod;
        }
    }
    long long answer = 1 % mod;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
            while (a[j][i])
            {
                long long quotient = a[i][i] / a[j][i];
                for (int k = i; k < n; k++)
                {
                    __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
                    value %= mod;
                    if (value < 0) value += mod;
                    a[i][k] = (long long)value;
                }
                swap(a[i], a[j]);
                answer = answer ? mod - answer : 0;
            }
        if (a[i][i] == 0) return 0;
        answer = (__int128)answer * a[i][i] % mod;
    }
    return answer;
}
```

模板题使用：Library Checker matrix_det_arbitrary_mod

题意：给方阵和模数，求行列式的最小非负余数，模数可以为合数。函数无需模逆元，按值接收矩阵，示例用 move 交出数据。0 阶方阵的行列式为 1，再对模数取余。

所需模板：determinant_mod（第 192 页，C.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/matrix_det_arbitrary_mod

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long mod;
    cin >> n >> mod;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto &row : a)
        for (auto &x : row) cin >> x;
    cout << determinant_mod(move(a), mod) << '\n';
}
```

附录 D　并查集

D.1　并查集

D.1.1　并查集

dsu(n)，点号 0..n-1，n≥0。find(x) 路径减半；same(x,y) 判断同集合；merge(x,y) 把 y 的根接到 x 的根，已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组，组内点号递增、组间按当前根编号排列；可能压缩路径，但不改变分组。fa、sz 可读，sz 仅根处有效；不要外部改写。本版只做路径减半，不按大小或秩合并；单次 find 最坏 O(n)。空间 O(n)，groups 另需 O(n) 输出与工作空间，并执行 n 次 find。空结构可调用 groups，不能查询顶点。

```
struct dsu
{
    vector<int> fa, sz;

    dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }

    int find(int x)
    {
        while (x != fa[x]) x = fa[x] = fa[fa[x]];
        return x;
    }

    bool same(int x, int y) { return find(x) == find(y); }

    bool merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y) return false;
        sz[x] += sz[y], fa[y] = x;
        return true;
    }

    int size(int x) { return sz[find(x)]; }

    vector<vector<int>> groups()
    {
        vector<vector<int>> res(fa.size());
        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
        res.erase(
            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
            res.end());
        return res;
    }
};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 1..n，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu（第 194 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3367

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    dsu d(n);
    while (m--)
    {
        int op, x, y;
        scanf("%d%d%d", &op, &x, &y);
        --x, --y;
        if (op == 1)
            d.merge(x, y);
        else
            puts(d.same(x, y) ? "Y" : "N");
    }
}
```

附录 E　树状数组

E.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。$n \le 100000$，$|x| \le 10000000$；总复杂度 $O(n \log n)$，空间 $O(n)$。

所需模板：Fenwick（第 196 页，E.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 F　线段树

F.1　线段树基础

F.1.1　通用线段树与左右边界搜索

segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 198 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

算法索引

Arborescence, 71

BellmanFord, 46
BiconnectedCore, 87
Binomial, 191
BipartiteMatching, 148
block_cut_forest, 96
Blossom, 155
BoundedCirculation, 139
BoundedMaxFlow, 141
bridge_component_forest, 90

Cactus, 91
Cayley formula, 169
CentroidDiameter, 37
CentroidPairs, 29
CentroidSum, 32
count_four_cycles, 97

dag_path_determinant, 180
determinant_mod, 192
Dijkstra, 45
Dinic, 123
DirectedEuler, 109
DominatorTree, 182
dsu, 194
DynamicDiameter, 3

enumerate_triangles, 98

Fenwick, 196
Floyd, 50
FunctionalGraph, 81

gomory_hu, 130

HLD, 19
HLPP, 118
Hungarian, 153

independent_set, 153

Kruskal, 67
KruskalTree, 57
KShortestWalks, 55

LiftingLCA, 11

LongChain, 14
Lucas, 189

Manhattan MST, 71
ManhattanMST, 60
matching_edges, 152
MatrixTreeMod, 176
maximum_closure, 128
MinCostFlow, 132
mincut_edges, 129
MinCycle, 102
minimum spanning tree, 71
minimum_mean_cycle, 100
mixed_euler_orientation, 115
mixed_euler_trail, 117
ModInt, 186

NegativeCostFlow, 136

path_intersection, 10
Prim, 69
Prüfer sequence, 169

regular_matching, 145

SecondMST, 63
segtree, 198
Spfa, 48
SteinerTree, 75
StoerWagner, 142
StrictSecondShortest, 53
SubtreeColors, 24

TarjanSCC, 84
TreeDiameter, 8
TreeIsomorphism, 42
tutte, 171
TwoSAT, 107

UndirectedEuler, 112

VirtualTree, 26

WeightedBlossom, 159

开放时间最短路, 52

树上路径交, 14

贪吃蛇, 52